

Complete Course: Machine Learning

From Fundamentals to Advanced Techniques

Data Science and Machine Learning Mastery

Contents

1	Introduction to Machine Learning	4
1.1	What is Machine Learning?	4
1.2	Types of Machine Learning	4
1.3	The Machine Learning Pipeline	5
2	Data Preprocessing	5
2.1	Data Types	5
2.2	Handling Missing Values	6
2.3	Feature Scaling	7
2.4	Encoding Categorical Variables	7
2.4.1	Choosing the Right Encoding Method	13
2.4.2	Best Practices for Categorical Encoding	13
2.5	Feature Selection Methods	15
3	Supervised Learning	15
3.1	Regression	15
3.1.1	Linear Regression	15
3.1.2	Regularization Techniques	20
3.2	Classification	20
3.2.1	Logistic Regression	21
3.2.2	Decision Trees for Classification	23
3.2.3	Random Forests	27
3.2.4	Gradient Boosting Machines	30
3.2.5	Mathematical Foundation	31
3.2.6	Support Vector Machines	34
3.3	Neural Networks	37
3.3.1	Backpropagation Algorithm	39
3.3.2	Python Implementation with PyTorch	39
3.3.3	Advanced Architectures	44

4	Unsupervised Learning	47
4.1	Clustering	47
4.1.1	K-Means Clustering	47
4.1.2	Mathematical Foundation	48
4.1.3	Hierarchical Clustering	52
4.1.4	DBSCAN (Density-Based Spatial Clustering)	55
4.2	Dimensionality Reduction	59
4.2.1	Principal Component Analysis (PCA)	59
5	Model Evaluation and Validation	63
5.1	Cross-Validation	63
5.2	Evaluation Metrics	65
5.2.1	Classification Metrics	65
5.2.2	Regression Metrics	66
5.2.3	Clustering Metrics	66
5.3	Python Implementation for Evaluation	66
6	Hyperparameter Tuning	72
6.0.1	Grid Search	72
6.0.2	Randomized Search	73
6.0.3	Bayesian Optimization	74
7	Advanced Topics and Best Practices	75
7.1	Handling Imbalanced Data	75
7.2	Feature Engineering Best Practices	77
7.3	Time Series Forecasting	79
8	Model Deployment	87
8.1	Model Serialization	87
8.2	Model Monitoring	87
9	Best Practices and Tips	95
9.1	Key Takeaways	95
9.2	Choosing the Right Algorithm	96
9.3	Common Pitfalls and Solutions	96
9.4	Performance Optimization Tips	97
9.5	Model Interpretability	97

1 Introduction to Machine Learning

1.1 What is Machine Learning?

Definition 1.1 (Machine Learning). *Machine Learning is the study of algorithms that improve automatically through experience. It is a subset of artificial intelligence that enables computers to learn from data without being explicitly programmed.*

Machine Learning is not about creating intelligent machines in the science fiction sense. It's about creating systems that can **learn from experience**. The key insight is that instead of explicitly programming rules, we create algorithms that can automatically improve their performance by analyzing data.

The Learning Paradigm: Traditional programming requires us to specify exact rules: "If condition X, then do Y." Machine Learning flips this: we give the system examples of inputs and desired outputs, and it *discovers* the rules itself. This is particularly powerful for problems where:

- The rules are too complex for humans to articulate (e.g., recognizing faces)
- The rules change over time (e.g., stock market patterns)
- We have massive amounts of data that contains patterns we haven't discovered yet

1.2 Types of Machine Learning

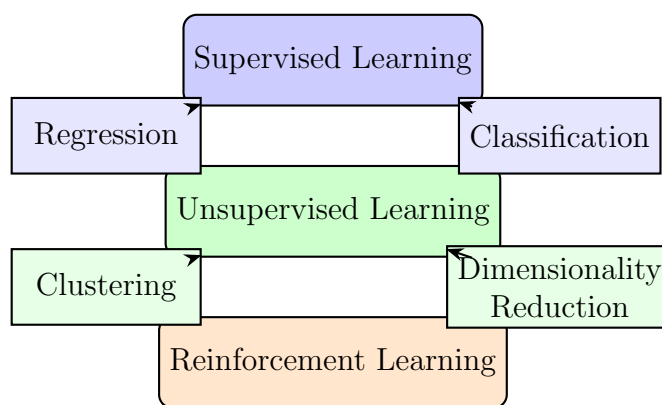


Figure 1: Main types of machine learning and their subtypes

The Learning Paradigm: Traditional programming requires us to specify exact rules: "If condition X, then do Y." Machine Learning flips this: we give the system examples of inputs and desired outputs, and it *discovers* the rules itself. This is particularly powerful for problems where:

- The rules are too complex for humans to articulate (e.g., recognizing faces)
- The rules change over time (e.g., stock market patterns)

- We have massive amounts of data that contains patterns we haven't discovered yet
1. **Supervised Learning:** Learn from labeled data
 2. **Unsupervised Learning:** Discover patterns in unlabeled data
 3. **Semi-Supervised Learning:** Combination of labeled and unlabeled data
 4. **Reinforcement Learning:** Learn through rewards and penalties
 5. **Self-Supervised Learning:** Generate labels from data itself

1.3 The Machine Learning Pipeline

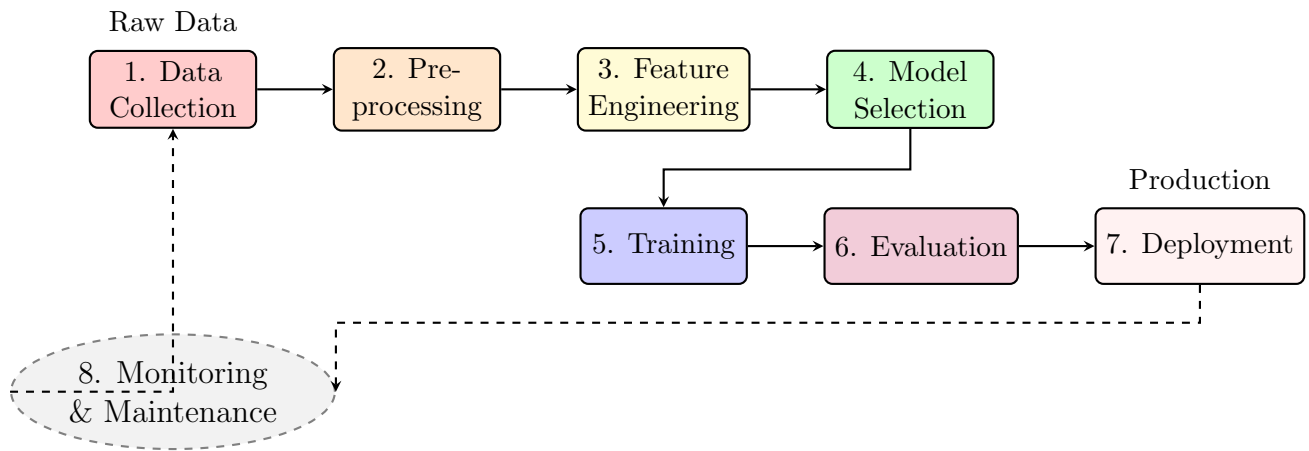


Figure 2: Complete machine learning pipeline with feedback loop

2 Data Preprocessing

2.1 Data Types

Definition 2.1 (Types of Features). • **Numerical:** Continuous or discrete numbers

- **Continuous:** Real numbers (e.g., temperature, price)
- **Discrete:** Integer values (e.g., age, count)

- **Categorical:** Limited number of possible values

- **Nominal:** No order (e.g., colors, cities)
- **Ordinal:** Has order (e.g., ratings, education level)

- **Text:** Free-form text data

- **Time Series:** Data points indexed in time order

- **Image:** Pixel values representing images

2.2 Handling Missing Values

Method	Description	When to Use
Mean/Median Imputation	Replace with mean/median	Numerical data, small missing rate
Mode Imputation	Replace with most frequent value	Categorical data
KNN Imputation	Use k-nearest neighbors	Relationships exist in data
Multiple Imputation	Create multiple imputed datasets	Statistical inference needed
Deletion	Remove rows/columns	Large dataset, small missing rate

Table 1: Methods for handling missing values

Missing Data Mechanisms:

- **MCAR (Missing Completely at Random):** Missingness unrelated to data
- **MAR (Missing at Random):** Missingness related to observed data
- **MNAR (Missing Not at Random):** Missingness related to unobserved data

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.impute import SimpleImputer, KNNImputer
4 from sklearn.experimental import enable_iterative_imputer
5 from sklearn.impute import IterativeImputer
6
7 # Sample data with missing values
8 data = pd.DataFrame({
9     'Age': [25, 30, np.nan, 35, 40, np.nan, 28],
10    'Salary': [50000, 60000, 55000, np.nan, 70000, 65000, 58000],
11    'Department': ['Sales', 'IT', 'IT', np.nan, 'Sales', 'IT', 'HR']
12 })
13
14 print("Original data with missing values:")
15 print(data)
16
17 # 1. Simple imputation
18 print("\n1. Mean imputation for Age:")
19 imputer_mean = SimpleImputer(strategy='mean')
20 data['Age_mean'] = imputer_mean.fit_transform(data[['Age']])
21
22 # 2. KNN imputation
23 print("\n2. KNN imputation:")
24 imputer_knn = KNNImputer(n_neighbors=2)
25 age_salary_imputed = imputer_knn.fit_transform(data[['Age', 'Salary']])
26 print(f"KNN imputed Age-Salary: {age_salary_imputed}")
27
```

```

28 # 3. Iterative imputation (MICE)
29 print("\n3. Iterative imputation (MICE):")
30 imputer_iter = IterativeImputer(max_iter=10, random_state=42)
31 iterative_imputed = imputer_iter.fit_transform(data[['Age', 'Salary']])
32 print(f"Iterative imputed: {iterative_imputed}")
33
34 # 4. Adding missing indicator
35 print("\n4. Adding missing indicators:")
36 data['Age_missing'] = data['Age'].isnull().astype(int)
37 data['Salary_missing'] = data['Salary'].isnull().astype(int)
38 print("Data with missing indicators:")
39 print(data[['Age', 'Age_missing', 'Salary', 'Salary_missing']])

```

Listing 1: Advanced Missing Value Handling in Python

2.3 Feature Scaling

Definition 2.2 (Standardization).

$$x' = \frac{x - \mu}{\sigma}$$

where μ is mean, σ is standard deviation. Results in $\mu = 0$, $\sigma = 1$.

Definition 2.3 (Min-Max Normalization).

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Scales to $[0, 1]$ range.

Definition 2.4 (Robust Scaling).

$$x' = \frac{x - \text{median}(x)}{IQR(x)}$$

Uses median and IQR, robust to outliers.

2.4 Encoding Categorical Variables

Categorical variables are non-numerical features that take on a limited number of values. Since most machine learning algorithms require numerical input, we need to encode categorical variables into numerical form.

1. **One-Hot Encoding:** Create binary columns for each category

- **Principle:** Create new binary columns (0/1) for each unique category
- **When to use:** Nominal variables (no order), small number of categories (≤15)
- **Advantages:** No ordinal relationship assumption, works well for linear models

- **Disadvantages:** High dimensionality (curse of dimensionality), sparse matrix

Example 2.5 (One-Hot Encoding). *For categorical feature "Color" with values: ["Red", "Blue", "Green"]*

Original	Color_Red	Color_Blue	Color_Green
Red	1	0	0
Blue	0	1	0
Green	0	0	1
Red	1	0	0

Table 2: One-hot encoding example

```

1  import pandas as pd
2  from sklearn.preprocessing import OneHotEncoder
3
4  # Sample data
5  data = pd.DataFrame({
6      'Color': ['Red', 'Blue', 'Green', 'Red', 'Blue'],
7      'Size': ['S', 'M', 'L', 'M', 'S']
8  })
9
10 print("Original data:")
11 print(data)
12
13 # Using pandas
14 data_encoded = pd.get_dummies(data, columns=['Color', 'Size'])
15 print("\nOne-hot encoded with pandas:")
16 print(data_encoded)
17
18 # Using scikit-learn
19 encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
20 encoded_array = encoder.fit_transform(data[['Color']])
21 print("\nOne-hot encoded with scikit-learn:")
22 print(encoded_array)
23 print("Categories:", encoder.categories_)

```

Listing 2: One-Hot Encoding in Python

2. Label Encoding: Assign integer to each category

- **Principle:** Assign a unique integer to each category
- **When to use:** Ordinal variables (natural order), tree-based models
- **Advantages:** Compact representation, maintains order for ordinal data
- **Disadvantages:** Creates artificial ordinal relationship (problematic for nominal)

Example 2.6 (Label Encoding). For categorical feature "Education" with values: ["High School", "Bachelor", "Master", "PhD"]

Original	Encoded
High School	0
Bachelor	1
Master	2
PhD	3
High School	0

Table 3: Label encoding example

```

1  from sklearn.preprocessing import LabelEncoder
2  import pandas as pd
3
4  # Sample data
5  data = pd.DataFrame({
6      'Education': ['High School', 'Bachelor', 'Master', 'PhD', 'High
      ↪ School'],
7      'Size': ['S', 'M', 'L', 'XL', 'M']
8  })
9
10 # Label encoding
11 label_encoder = LabelEncoder()
12 data['Education_encoded'] = label_encoder.fit_transform(data['Education'
13 ↪ ])
14
15 print("Label encoded data:")
16 print(data[['Education', 'Education_encoded']])
17 print("\nMapping:", dict(zip(label_encoder.classes_,
18                               range(len(label_encoder.classes_))))))
19
20 # For ordinal data with custom order
21 size_order = ['XS', 'S', 'M', 'L', 'XL']
22 data['Size_encoded'] = data['Size'].apply(lambda x: size_order.index(x))
23 print("\nOrdinal encoding with custom order:")
24 print(data[['Size', 'Size_encoded']])

```

Listing 3: Label Encoding in Python

3. Target Encoding: Encode with target mean

- **Principle:** Replace category with mean target value for that category
- **When to use:** High cardinality features, tree-based models
- **Advantages:** Captures relationship with target, reduces dimensionality

- **Disadvantages:** Risk of overfitting, requires careful validation

Example 2.7 (Target Encoding). *For categorical feature "City" and target "Price":*

City	Price	City_Encoded
NYC	100	105
LA	110	107.5
NYC	110	105
Chicago	90	95
LA	105	107.5
Chicago	100	95

Table 4: Target encoding example (means: NYC=105, LA=107.5, Chicago=95)

```

1  import pandas as pd
2  import numpy as np
3  from sklearn.model_selection import KFold
4
5  # Sample data
6  data = pd.DataFrame({
7      'City': ['NYC', 'LA', 'NYC', 'Chicago', 'LA', 'Chicago'],
8      'Price': [100, 110, 110, 90, 105, 100]
9  })
10
11 # Simple target encoding (leaky - don't use this in practice)
12 city_means = data.groupby('City')['Price'].mean()
13 data['City_encoded_simple'] = data['City'].map(city_means)
14
15 print("Simple target encoding (demonstration only - has data leakage):")
16 print(data[['City', 'Price', 'City_encoded_simple']])
17
18 # Proper target encoding with cross-validation
19 def target_encode_with_cv(df, col, target, n_splits=5):
20     """
21     Target encoding with cross-validation to prevent leakage.
22     """
23     df = df.copy()
24     df[f'{col}_encoded'] = 0
25
26     kf = KFold(n_splits=n_splits, shuffle=True, random_state=42)
27
28     for train_idx, val_idx in kf.split(df):
29         X_train, X_val = df.iloc[train_idx], df.iloc[val_idx]
30
31         # Calculate mean target for each category in training fold
32         means = X_train.groupby(col)[target].mean()

```

```

33         # Map to validation fold
34         df.loc[val_idx, f'{col}_encoded'] = X_val[col].map(means)
35
36     # Fill NaN (categories not in training)
37     global_mean = df[target].mean()
38     df[f'{col}_encoded'] = df[f'{col}_encoded'].fillna(global_mean)
39
40
41     return df[f'{col}_encoded']
42
43 data['City_encoded_cv'] = target_encode_with_cv(data, 'City', 'Price')
44 print("\nTarget encoding with cross-validation:")
45 print(data[['City', 'Price', 'City_encoded_cv']])

```

Listing 4: Target Encoding in Python

4. Frequency Encoding: Encode with category frequency

- **Principle:** Replace category with its frequency in the dataset
- **When to use:** High cardinality features, tree-based models
- **Advantages:** No new dimensions, handles unseen categories
- **Disadvantages:** Loses category identity, same frequency = same encoding

Example 2.8 (Frequency Encoding). *For categorical feature "Country":*

Country	Count	Frequency_Encoded
USA	3	0.5 (3/6)
Canada	2	0.333 (2/6)
Mexico	1	0.167 (1/6)
USA	3	0.5
Canada	2	0.333
USA	3	0.5

Table 5: Frequency encoding example (total=6 samples)

```

1  import pandas as pd
2
3  # Sample data
4  data = pd.DataFrame({
5      'Country': ['USA', 'Canada', 'Mexico', 'USA', 'Canada', 'USA'],
6      'Value': [10, 20, 30, 40, 50, 60]
7  })
8
9  # Frequency encoding
10 freq = data['Country'].value_counts(normalize=True)

```

```

11 data['Country_freq'] = data['Country'].map(freq)
12
13 # Count encoding (alternative)
14 count = data['Country'].value_counts()
15 data['Country_count'] = data['Country'].map(count)
16
17 print("Frequency encoding:")
18 print(data[['Country', 'Country_freq', 'Country_count']])
19 print("\nFrequency mapping:")
20 print(freq)
21
22 # Handle unseen categories
23 test_data = pd.DataFrame({'Country': ['USA', 'Canada', 'France']})
24 test_data['Country_freq'] = test_data['Country'].map(freq).fillna(0)
25 print("\nTest data with unseen category (France):")
26 print(test_data)

```

Listing 5: Frequency Encoding in Python

5. Bonus: Binary Encoding - Compact representation for high cardinality

- **Principle:** First convert to ordinal, then to binary, then split bits
- **When to use:** High cardinality categorical variables (>15 categories)
- **Advantages:** More compact than one-hot, maintains some information

```

1 import category_encoders as ce
2 import pandas as pd
3
4 # Sample high cardinality data
5 data = pd.DataFrame({
6     'City': ['New York', 'Los Angeles', 'Chicago', 'Houston',
7             'Phoenix', 'Philadelphia', 'San Antonio', 'San Diego',
8             'Dallas', 'San Jose'] * 3, # 30 samples
9     'Value': range(30)
10 })
11
12 # Binary encoding
13 encoder = ce.BinaryEncoder(cols=['City'])
14 data_encoded = encoder.fit_transform(data)
15
16 print("Original data shape:", data.shape)
17 print("Binary encoded shape:", data_encoded.shape)
18 print("\nFirst few rows of binary encoded data:")
19 print(data_encoded[['City_0', 'City_1', 'City_2', 'City_3']].head())
20
21 # Compare with one-hot
22 data_onehot = pd.get_dummies(data, columns=['City'])

```

```

23 print("\nOne-hot encoded shape:", data_onehot.shape)
24 print("Binary encoding reduces dimensions from", data_onehot.shape[1],
25       "to", data_encoded.shape[1])

```

Listing 6: Binary Encoding in Python

2.4.1 Choosing the Right Encoding Method

Method	Best For	Cardinality	Models
One-Hot	Nominal, few categories	Low (≤15)	Linear models, Neural nets
Label	Ordinal	Any	Tree-based models
Target	High cardinality	High	Tree-based, Gradient boosting
Frequency	High cardinality	High	Tree-based models
Binary	High cardinality	High	Most models

Table 6: Choosing categorical encoding methods

2.4.2 Best Practices for Categorical Encoding

1. **Always split data before encoding:** Avoid data leakage
2. **Handle unseen categories:** Use special encoding or imputation
3. **For tree-based models:** Label encoding often works well
4. **For linear models:** One-hot encoding preferred
5. **High cardinality:** Use target, frequency, or binary encoding
6. **Ordinal variables:** Use label encoding with custom order
7. **Test multiple methods:** Different encodings work better for different problems

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.model_selection import train_test_split
4 from sklearn.preprocessing import OneHotEncoder, LabelEncoder
5 from sklearn.compose import ColumnTransformer
6 from sklearn.pipeline import Pipeline
7 from sklearn.ensemble import RandomForestClassifier
8
9 # Sample data with mixed types
10 data = pd.DataFrame({
11     'City': ['NYC', 'LA', 'Chicago', 'NYC', 'LA', 'Chicago'] * 10,
12     'Size': ['S', 'M', 'L', 'M', 'L', 'S'] * 10,
13     'Price': np.random.randint(50, 200, 60),

```

```

14     'Target': np.random.randint(0, 2, 60)
15 })
16
17 # Split data
18 X = data.drop('Target', axis=1)
19 y = data['Target']
20 X_train, X_test, y_train, y_test = train_test_split(
21     X, y, test_size=0.2, random_state=42
22 )
23
24 # Create preprocessing pipeline
25 preprocessor = ColumnTransformer(
26     transformers=[
27         # One-hot for nominal with few categories
28         ('city', OneHotEncoder(handle_unknown='ignore'), ['City']),
29         # Label encoding for ordinal
30         ('size', 'passthrough', ['Size']) # Will convert manually
31     ],
32     remainder='passthrough'
33 )
34
35 # Convert size to ordinal manually
36 size_order = {'S': 0, 'M': 1, 'L': 2}
37 X_train['Size'] = X_train['Size'].map(size_order)
38 X_test['Size'] = X_test['Size'].map(size_order).fillna(0) # Handle unseen
39
40 # Create and train pipeline
41 pipeline = Pipeline([
42     ('preprocessor', preprocessor),
43     ('classifier', RandomForestClassifier(n_estimators=100, random_state=42))
44 ])
45
46 pipeline.fit(X_train, y_train)
47 score = pipeline.score(X_test, y_test)
48 print(f"Model accuracy with proper encoding: {score:.4f}")

```

Listing 7: Complete Encoding Pipeline

2.5 Feature Selection Methods

Method	Description	Best for
Variance Threshold	Remove low-variance features	All types
Correlation Analysis	Remove highly correlated features	Numerical
Chi-Square Test	Test independence with target	Categorical
Mutual Information	Measure dependency	All types
LASSO	L1 regularization for selection	Linear models
Tree-based Importance	Use feature importance from trees	Tree models

Table 7: Feature selection methods

3 Supervised Learning

Think of it as: A student learning with a teacher who provides answers. We have input-output pairs, and the algorithm learns the mapping.

When to use it: When you have labeled data and want to predict something. Examples:

- **Classification:** Is this email spam or not? (Discrete output)
- **Regression:** What will the temperature be tomorrow? (Continuous output)

Key Characteristics:

- Requires labeled training data
- Goal is to predict output for new inputs
- Evaluated by comparing predictions to true labels

3.1 Regression

Purpose: Predict continuous numerical values. Examples: house prices, temperature, stock prices.

3.1.1 Linear Regression

The Intuition: Find the best-fitting straight line through data points. Imagine plotting house sizes against prices and drawing a line that minimizes the total distance from all points to the line.



Figure 3: Linear regression minimizes the vertical distances (errors) between points and the line

Mathematical Foundation:

Definition 3.1 (Simple Linear Regression). *Given n data points (x_i, y_i) , find parameters β_0 (intercept) and β_1 (slope) that minimize:*

$$J(\beta_0, \beta_1) = \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \frac{1}{2n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

*This is called the **Mean Squared Error (MSE)** loss function.*

Algorithm 1 Gradient Descent for Linear Regression

```
1: procedure GRADIENTDESCENT( $X, y, \alpha, iterations$ )
2:   Initialize  $\beta_0, \beta_1$  to small random values
3:   for  $iter = 1$  to  $iterations$  do
4:     Compute predictions:  $\hat{y} = \beta_0 + \beta_1 X$ 
5:     Compute errors:  $error = \hat{y} - y$ 
6:     Compute gradient for  $\beta_0$ :  $\frac{\partial J}{\partial \beta_0} = \frac{1}{n} \sum_{i=1}^n error_i$ 
7:     Compute gradient for  $\beta_1$ :  $\frac{\partial J}{\partial \beta_1} = \frac{1}{n} \sum_{i=1}^n (error_i \cdot X_i)$ 
8:     Update  $\beta_0$ :  $\beta_0 \leftarrow \beta_0 - \alpha \cdot \frac{\partial J}{\partial \beta_0}$ 
9:     Update  $\beta_1$ :  $\beta_1 \leftarrow \beta_1 - \alpha \cdot \frac{\partial J}{\partial \beta_1}$ 
10:    if change in  $J$  is very small then break
11:    end if
12:  end for
13:  return  $\beta_0, \beta_1$ 
14: end procedure
```

How It's Learned - Gradient Descent:

Learning Rate α Explained: The learning rate controls step size:

- Too large: May overshoot the minimum, fail to converge
- Too small: Takes too long to converge
- Typical values: 0.01, 0.001, 0.0001

Multiple Linear Regression: When we have multiple features:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \epsilon$$

In matrix form: $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$

Theorem 3.2 (Normal Equation Solution). *For linear regression without regularization:*

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

This gives the exact solution but is computationally expensive for large datasets ($O(n^3)$).

Assumptions and Limitations:

1. **Linearity:** Relationship between features and target is linear
2. **Independence:** Observations are independent of each other
3. **Homoscedasticity:** Constant variance of errors
4. **Normality:** Errors are normally distributed

5. **No multicollinearity:** Features aren't highly correlated

When linear regression fails:

- Non-linear relationships
- Outliers (sensitive to them)
- High-dimensional data (curse of dimensionality)

How Parameters are Learned: Gradient Descent

Algorithm 2 Gradient Descent for Linear Regression

```
1: procedure GRADIENTDESCENT( $X, y, \alpha, iterations$ )
2:   Initialize  $\beta_0, \beta_1$  randomly or to zero
3:   for  $iter = 1$  to  $iterations$  do
4:     Compute predictions:  $\hat{y} = \beta_0 + \beta_1 X$ 
5:     Compute error:  $error = \hat{y} - y$ 
6:     Update  $\beta_0$ :  $\beta_0 \leftarrow \beta_0 - \alpha \cdot \frac{1}{n} \sum_{i=1}^n error_i$ 
7:     Update  $\beta_1$ :  $\beta_1 \leftarrow \beta_1 - \alpha \cdot \frac{1}{n} \sum_{i=1}^n (error_i \cdot X_i)$ 
8:     if convergence criteria met then
9:       break
10:    end if
11:  end for
12:  return  $\beta_0, \beta_1$ 
13: end procedure
```

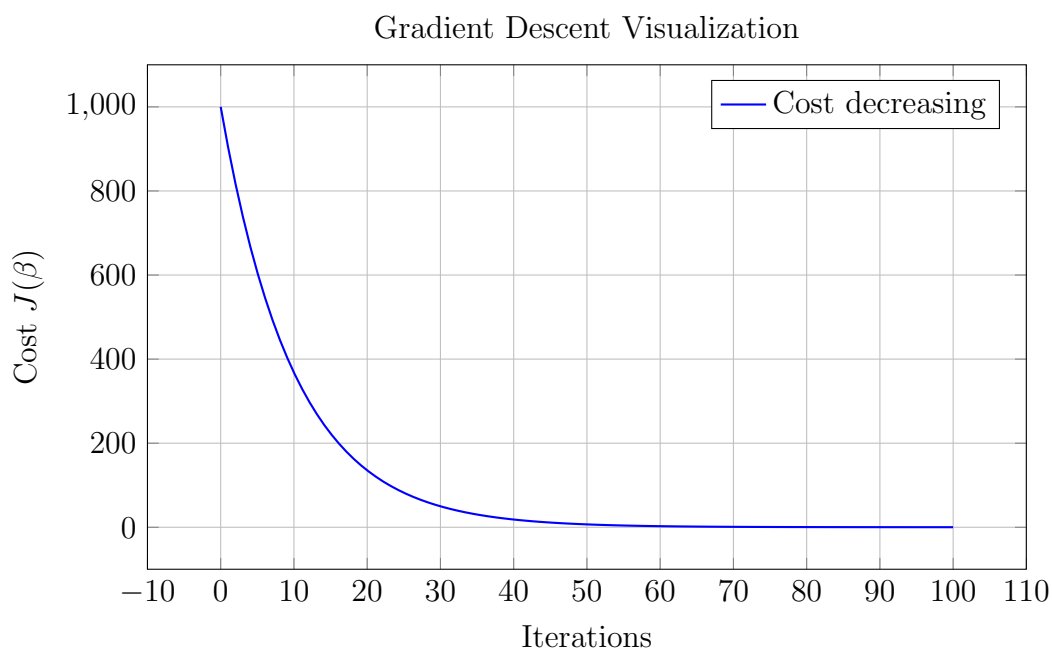


Figure 4: Gradient descent reduces cost over iterations

Learning rate α explained: The learning rate controls step size. Too large (α large): overshoots minimum. Too small (α small): slow convergence. Typical values: 0.01, 0.001, 0.0001.

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.linear_model import LinearRegression, Ridge, Lasso
4 from sklearn.model_selection import train_test_split
5 from sklearn.metrics import mean_squared_error, r2_score
6 from sklearn.preprocessing import StandardScaler
7
8 # Generate sample data
9 np.random.seed(42)
10 n_samples = 1000
11 X = np.random.randn(n_samples, 3) # 3 features
12 # Create target with linear relationship plus noise
13 y = 2.5 + 1.5*X[:,0] + 0.8*X[:,1] - 1.2*X[:,2] + np.random.randn(n_samples)*0.5
14
15 # Split data
16 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
17     ↪ random_state=42)
18
19 # Scale features
20 scaler = StandardScaler()
21 X_train_scaled = scaler.fit_transform(X_train)
22 X_test_scaled = scaler.transform(X_test)
23
24 # Train models
25 models = {
26     'Linear Regression': LinearRegression(),
27     'Ridge Regression': Ridge(alpha=1.0),
28     'Lasso Regression': Lasso(alpha=0.1)
29 }
30
31 for name, model in models.items():
32     model.fit(X_train_scaled, y_train)
33     y_pred = model.predict(X_test_scaled)
34
35     mse = mean_squared_error(y_test, y_pred)
36     r2 = r2_score(y_test, y_pred)
37
38     print(f"{name}:")
39     print(f" MSE: {mse:.4f}")
40     print(f" R2 Score: {r2:.4f}")
41
42     if hasattr(model, 'coef_'):
43         print(f" Coefficients: {model.coef_}")
44         print(f" Intercept: {model.intercept_:.4f}")
```

Listing 8: Linear Regression Implementation

3.1.2 Regularization Techniques

Why Regularization? When models become too complex, they can overfit - learn noise in the training data instead of the underlying pattern. Regularization adds constraints to prevent this.

Definition 3.3 (Ridge Regression (L2 Regularization)). *Adds penalty proportional to square of coefficients:*

$$J(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

Effect: *Shrinks coefficients toward zero but doesn't eliminate any.*

Definition 3.4 (LASSO Regression (L1 Regularization)). *Adds penalty proportional to absolute value of coefficients:*

$$J(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

Effect: *Can shrink coefficients exactly to zero - performs feature selection.*

Definition 3.5 (Elastic Net). *Combines L1 and L2 regularization:*

$$J(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda_1 \sum_{j=1}^p |\beta_j| + \lambda_2 \sum_{j=1}^p \beta_j^2$$

Effect: *Gets benefits of both Ridge and LASSO.*

Choosing λ (Regularization Strength):

- Large λ : Strong regularization, simpler model
- Small λ : Weak regularization, complex model
- Use cross-validation to find optimal λ

3.2 Classification

Purpose: Predict discrete categories. Examples: spam detection, disease diagnosis, image recognition.

3.2.1 Logistic Regression

The Intuition: Despite its name, logistic regression is for classification! It models the probability that an instance belongs to a particular class. Think of it as "How likely is this email to be spam?"

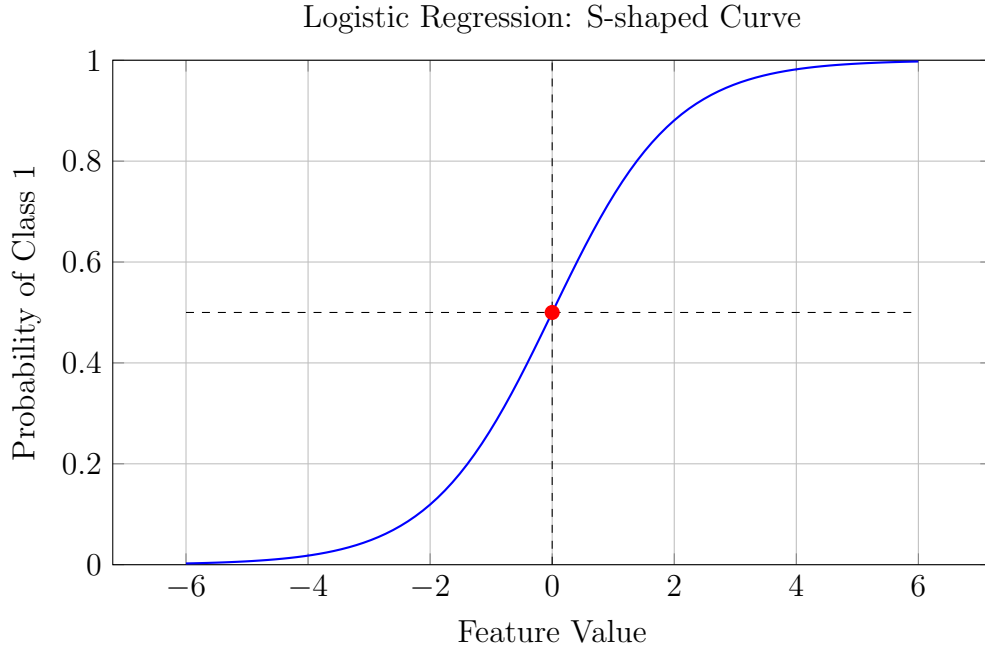


Figure 5: Logistic function maps any input to probability between 0 and 1

Mathematical Foundation:

Definition 3.6 (Logistic (Sigmoid) Function).

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

This S-shaped curve maps any real number to $(0, 1)$ range, perfect for probabilities.

For binary classification:

$$P(Y = 1|X) = \sigma(\beta^\top \mathbf{x}) = \frac{1}{1 + e^{-\beta^\top \mathbf{x}}}$$

$$P(Y = 0|X) = 1 - P(Y = 1|X)$$

Decision Boundary: Logistic regression makes predictions by:

$$\hat{y} = \begin{cases} 1 & \text{if } P(Y = 1|X) \geq 0.5 \\ 0 & \text{if } P(Y = 1|X) < 0.5 \end{cases}$$

This corresponds to $\beta^\top \mathbf{x} \geq 0$.

How It's Learned - Maximum Likelihood: Instead of minimizing squared error, logistic regression maximizes likelihood:

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^n [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

where $p_i = \sigma(\boldsymbol{\beta}^\top \mathbf{x}_i)$.

Algorithm 3 Logistic Regression Training with Gradient Descent

```

1: procedure LOGISTICGRADIENTDESCENT( $X, y, \alpha, iterations$ )
2:   Initialize  $\boldsymbol{\beta}$  to zeros or small random values
3:   for  $iter = 1$  to  $iterations$  do
4:     Compute probabilities:  $\mathbf{p} = \sigma(\mathbf{X}\boldsymbol{\beta})$ 
5:     Compute gradient:  $\nabla \ell = \mathbf{X}^\top (\mathbf{p} - \mathbf{y})$ 
6:     Update parameters:  $\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} - \alpha \nabla \ell$ 
7:     if converged then break
8:   end if
9: end for
10: return  $\boldsymbol{\beta}$ 
11: end procedure

```

Multiclass Logistic Regression: For $K > 2$ classes, use softmax function:

$$P(Y = k|X) = \frac{e^{\boldsymbol{\beta}_k^\top \mathbf{x}}}{\sum_{j=1}^K e^{\boldsymbol{\beta}_j^\top \mathbf{x}}}$$

This gives probability distribution over all classes.

```

1 import numpy as np
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.model_selection import train_test_split
4 from sklearn.datasets import load_iris
5 from sklearn.metrics import accuracy_score, confusion_matrix,
  ↳ classification_report
6 from sklearn.preprocessing import StandardScaler
7
8 # Load iris dataset (3 classes)
9 iris = load_iris()
10 X, y = iris.data, iris.target
11
12 # Split data
13 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
  ↳ random_state=42)
14
15 # Scale features (important for logistic regression)
16 scaler = StandardScaler()

```

```

17 X_train_scaled = scaler.fit_transform(X_train)
18 X_test_scaled = scaler.transform(X_test)
19
20 # Create and train logistic regression model
21 log_reg = LogisticRegression(
22     penalty='l2', # Regularization type
23     C=1.0, # Inverse regularization strength (smaller = stronger)
24     solver='lbfgs', # Optimization algorithm
25     multi_class='multinomial', # For multiclass
26     max_iter=1000,
27     random_state=42
28 )
29
30 log_reg.fit(X_train_scaled, y_train)
31
32 # Make predictions
33 y_pred = log_reg.predict(X_test_scaled)
34 y_pred_proba = log_reg.predict_proba(X_test_scaled)
35
36 # Evaluate
37 accuracy = accuracy_score(y_test, y_pred)
38 print(f"Accuracy: {accuracy:.4f}")
39 print("\nConfusion Matrix:")
40 print(confusion_matrix(y_test, y_pred))
41 print("\nClassification Report:")
42 print(classification_report(y_test, y_pred, target_names=iris.target_names))
43
44 # Show probabilities for first few test samples
45 print("\nPrediction probabilities for first 3 test samples:")
46 for i in range(3):
47     print(f"Sample {i}: True class={iris.target_names[y_test[i]]}")
48     for j, prob in enumerate(y_pred_proba[i]):
49         print(f" {iris.target_names[j]}: {prob:.4f}")
50     print()

```

Listing 9: Logistic Regression Implementation

3.2.2 Decision Trees for Classification

The Intuition: Ask a series of yes/no questions to classify instances. Like playing "20 Questions" - "Is the animal larger than a dog?", "Does it have feathers?", etc.

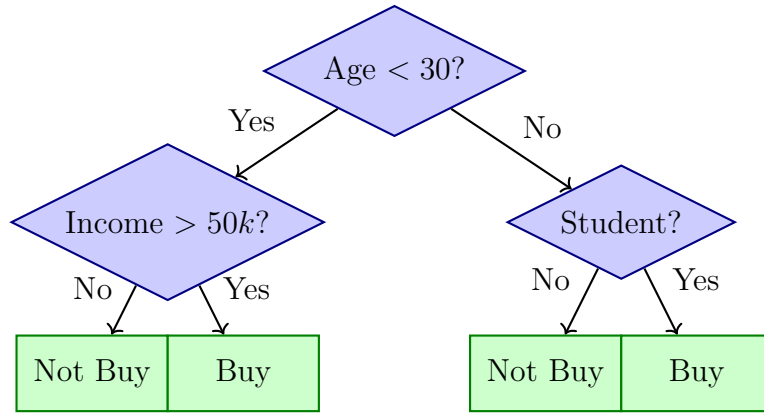


Figure 6: Decision tree example for product purchase prediction

Key Concepts:

- **Root Node:** First question/decision
- **Internal Nodes:** Subsequent decisions
- **Leaf Nodes:** Final predictions/classes
- **Branches:** Outcomes of decisions

Splitting Criteria: How to choose which feature to split on?

Definition 3.7 (Information Gain). *Based on entropy reduction:*

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

where $H(S)$ is entropy: $H(S) = -\sum_{k=1}^K p_k \log_2 p_k$

Definition 3.8 (Gini Impurity). *Measures how often a randomly chosen element would be incorrectly labeled:*

$$I_G(p) = 1 - \sum_{k=1}^K p_k^2$$

Lower Gini impurity = more pure node.

Algorithm 4 CART (Classification and Regression Trees) Algorithm

```
1: procedure CART( $D$ , depth, max_depth, min_samples_split)
2:   if all samples same class or depth = max_depth or  $|D| < \text{min\_samples\_split}$  then
3:     return leaf node with majority class
4:   end if
5:   Find best feature and split point that maximizes information gain
6:   Split  $D$  into  $D_{\text{left}}$  and  $D_{\text{right}}$ 
7:   left_child  $\leftarrow$  CART( $D_{\text{left}}$ , depth+1, max_depth, min_samples_split)
8:   right_child  $\leftarrow$  CART( $D_{\text{right}}$ , depth+1, max_depth, min_samples_split)
9:   return node(feature, split_point, left_child, right_child)
10: end procedure
```

Decision Tree Algorithm:**Strengths and Weaknesses: Strengths:**

- Easy to understand and interpret
- Can handle both numerical and categorical data
- Non-parametric (no assumptions about data distribution)

Weaknesses:

- Prone to overfitting (especially deep trees)
- Unstable (small data changes can change tree structure)
- Can create biased trees if classes are imbalanced

Pruning: To prevent overfitting:

- **Pre-pruning:** Stop growing tree early (max_depth, min_samples_leaf)
- **Post-pruning:** Grow full tree, then remove unnecessary branches

```
1 from sklearn.tree import DecisionTreeClassifier, plot_tree
2 from sklearn.datasets import load_breast_cancer
3 from sklearn.model_selection import train_test_split
4 import matplotlib.pyplot as plt
5
6 # Load data
7 data = load_breast_cancer()
8 X, y = data.data, data.target
9
10 # Split data
11 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪ random_state=42)
```

```

12
13 # Create decision tree with different parameters
14 trees = {
15     'Shallow (depth=3)': DecisionTreeClassifier(max_depth=3, random_state=42),
16     'Deep (depth=10)': DecisionTreeClassifier(max_depth=10, random_state=42),
17     'Pruned (min_samples_leaf=5)': DecisionTreeClassifier(min_samples_leaf=5,
18         ↪ random_state=42)
19 }
20 for name, tree in trees.items():
21     tree.fit(X_train, y_train)
22     train_acc = tree.score(X_train, y_train)
23     test_acc = tree.score(X_test, y_test)
24     print(f"{name}:")
25     print(f" Training Accuracy: {train_acc:.4f}")
26     print(f" Test Accuracy: {test_acc:.4f}")
27     print(f" Difference (overfitting): {train_acc - test_acc:.4f}")
28     print()
29
30 # Visualize one tree
31 plt.figure(figsize=(20, 10))
32 plot_tree(trees['Shallow (depth=3)'],
33     feature_names=data.feature_names,
34     class_names=data.target_names,
35     filled=True,
36     rounded=True,
37     fontsize=10)
38 plt.title("Decision Tree Visualization (Max Depth = 3)")
39 plt.show()
40
41 # Feature importance
42 importances = trees['Shallow (depth=3)'].feature_importances_
43 indices = np.argsort(importances)[::-1][:10] # Top 10 features
44
45 print("Top 10 important features:")
46 for i in range(10):
47     print(f" {i+1}. {data.feature_names[indices[i]]}: {importances[indices[i]]:.4f}
48     ↪ ")

```

Listing 10: Decision Tree Implementation

Regression Trees(Decision Trees for regression)

For regression, split based on MSE reduction:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y})^2$$

$$\Delta MSE = MSE(D) - \left(\frac{|D_{\text{left}}|}{|D|} MSE(D_{\text{left}}) + \frac{|D_{\text{right}}|}{|D|} MSE(D_{\text{right}}) \right)$$

3.2.3 Random Forests

The Intuition: "Wisdom of the crowd." Instead of asking one expert (decision tree), ask many experts and take a vote. Each expert sees slightly different data and focuses on different aspects.

Ensemble Learning Principle:

Weak Learners $\rightarrow$ Strong Learner

Many simple models (weak learners) combined create a powerful model (strong learner).

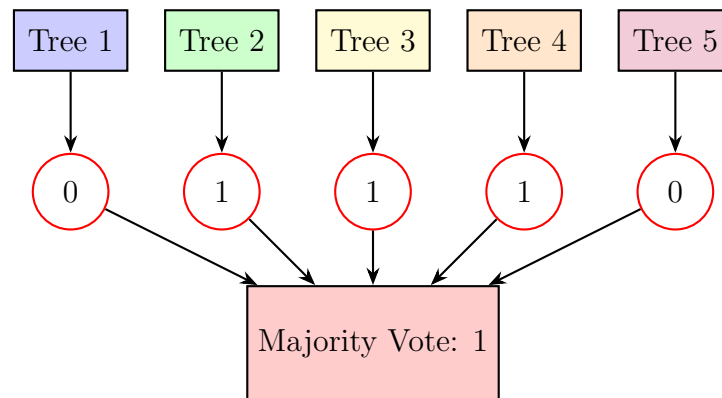


Figure 7: Random Forest: Ensemble of decision trees voting on final prediction

How Random Forests Work:

1. **Bootstrap Sampling:** Create multiple datasets by sampling with replacement
2. **Random Feature Selection:** At each split, consider only random subset of features
3. **Build Trees:** Grow decision trees on each bootstrap sample
4. **Aggregate:** For classification: majority vote; For regression: average

Algorithm 5 Random Forest Algorithm

```
1: procedure RANDOMFOREST( $D, B, m, n_{\min}$ )
2:   Initialize empty forest  $F$ 
3:   for  $b = 1$  to  $B$  do
4:     Sample  $D_b$  from  $D$  with replacement (bootstrap)
5:     Train decision tree  $T_b$  on  $D_b$  with:
6:       At each split: randomly select  $m$  features
7:       Split using best among  $m$  features
8:       Grow until  $n_{\min}$  samples in leaf
9:     Add  $T_b$  to  $F$ 
10:  end for
11:  return  $F$ 
12: end procedure
```

Key Parameters:

- **n_estimators**: Number of trees (more = better but slower)
- **max_features**: Number of features to consider at each split
- **max_depth**: Maximum depth of each tree
- **min_samples_split**: Minimum samples required to split a node

Out-of-Bag (OOB) Error: Special feature of random forests - can estimate error without separate validation set:

$$\text{OOB Error} = \frac{1}{n} \sum_{i=1}^n \mathbb{I}(\hat{y}_i^{\text{OOB}} \neq y_i)$$

where $\hat{y}_i^{\text{OOB}}$ is prediction using only trees that didn't include sample i in their bootstrap.

```
1 from sklearn.ensemble import RandomForestClassifier
2 from sklearn.datasets import make_classification
3 from sklearn.model_selection import train_test_split, GridSearchCV
4 import numpy as np
5
6 # Generate sample data
7 X, y = make_classification(n_samples=1000, n_features=20, n_informative=10,
8     ↪ random_state=42)
9
10 # Split data
11 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
12     ↪ random_state=42)
13
14 # Create Random Forest
15 rf = RandomForestClassifier(
16     n_estimators=100, # Number of trees
```

```

15     criterion='gini', # Splitting criterion
16     max_depth=None, # Grow trees fully
17     min_samples_split=2, # Minimum samples to split
18     min_samples_leaf=1, # Minimum samples in leaf
19     max_features='sqrt', # sqrt(n_features) considered at each split
20     bootstrap=True, # Use bootstrap samples
21     oob_score=True, # Calculate out-of-bag score
22     n_jobs=-1, # Use all CPU cores
23     random_state=42
24 )
25
26 rf.fit(X_train, y_train)
27
28 # Evaluate
29 train_acc = rf.score(X_train, y_train)
30 test_acc = rf.score(X_test, y_test)
31 oob_acc = rf.oob_score_
32
33 print(f"Training Accuracy: {train_acc:.4f}")
34 print(f"Test Accuracy: {test_acc:.4f}")
35 print(f"OOB Score: {oob_acc:.4f}")
36 print(f"Overfitting (train - test): {train_acc - test_acc:.4f}")
37
38 # Feature importance
39 importances = rf.feature_importances_
40 indices = np.argsort(importances)[::-1]
41
42 print("\nTop 10 Feature Importances:")
43 for i in range(min(10, len(importances))):
44     print(f" Feature {indices[i]}: {importances[indices[i]]:.4f}")
45
46 # Hyperparameter tuning
47 param_grid = {
48     'n_estimators': [50, 100, 200],
49     'max_depth': [None, 10, 20, 30],
50     'min_samples_split': [2, 5, 10],
51     'min_samples_leaf': [1, 2, 4],
52     'max_features': ['sqrt', 'log2', None]
53 }
54
55 # Use grid search to find best parameters
56 grid_search = GridSearchCV(
57     RandomForestClassifier(random_state=42),
58     param_grid,
59     cv=5,
60     scoring='accuracy',
61     n_jobs=-1,

```

```

62     verbose=1
63 )
64
65 grid_search.fit(X_train, y_train)
66
67 print(f"\nBest Parameters: {grid_search.best_params_}")
68 print(f"Best Cross-Validation Score: {grid_search.best_score_:.4f}")
69 print(f"Test Score with Best Parameters: {grid_search.score(X_test, y_test):.4f}"
    ↪ )

```

Listing 11: Random Forest Implementation

3.2.4 Gradient Boosting Machines

The Intuition: Learn from mistakes. Like a student who focuses more on questions they got wrong, GBM builds new models that correct errors of previous models.

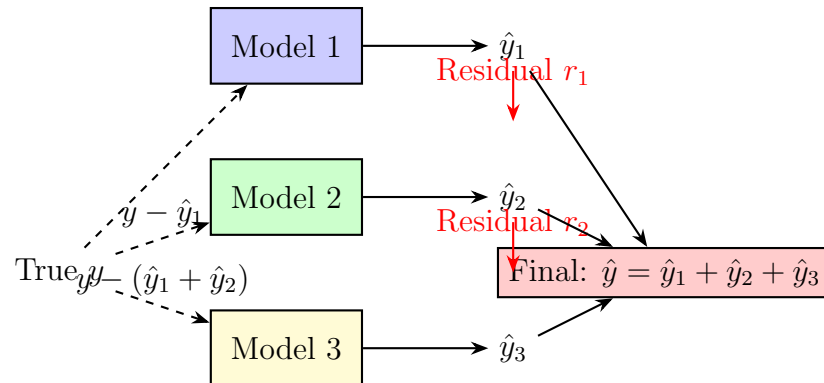


Figure 8: Gradient boosting: Each new model learns from residuals (errors) of previous models

How Gradient Boosting Works:

1. Start with initial prediction (e.g., mean of target values)
2. Calculate residuals (errors) of current model
3. Train new model to predict these residuals
4. Add this new model's predictions to ensemble (with small learning rate)
5. Repeat until stopping criterion met

Algorithm 6 Gradient Boosting Algorithm

```
1: procedure GRADIENTBOOSTING( $X, y, M, \nu$ )
2:   Initialize  $F_0(x) = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, \gamma)$ 
3:   for  $m = 1$  to  $M$  do
4:     Compute residuals:  $r_{im} = - \left[ \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}}$ 
5:     Fit weak learner  $h_m(x)$  to residuals  $\{(x_i, r_{im})\}$ 
6:     Compute multiplier:  $\gamma_m = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, F_{m-1}(x_i) + \gamma h_m(x_i))$ 
7:     Update:  $F_m(x) = F_{m-1}(x) + \nu \gamma_m h_m(x)$ 
8:   end for
9:   return  $F_M(x)$ 
10: end procedure
```

3.2.5 Mathematical Foundation

Definition 3.9 (Gradient Boosting). *Sequential ensemble method where each new model corrects errors of previous models.*

Algorithm 7 Gradient Boosting Algorithm

```
1: procedure GRADIENTBOOSTING( $D, M, \nu$ )
2:   Initialize  $F_0(\mathbf{x}) = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, \gamma)$ 
3:   for  $m = 1$  to  $M$  do
4:     Compute pseudo-residuals:  $r_{im} = - \left[ \frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right]_{F=F_{m-1}}$ 
5:     Fit weak learner  $h_m(\mathbf{x})$  to pseudo-residuals  $\{(\mathbf{x}_i, r_{im})\}_{i=1}^n$ 
6:     Compute multiplier:  $\gamma_m = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, F_{m-1}(\mathbf{x}_i) + \gamma h_m(\mathbf{x}_i))$ 
7:     Update:  $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu \gamma_m h_m(\mathbf{x})$ 
8:   end for
9:   return  $F_M(\mathbf{x})$ 
10: end procedure
```

XGBoost (Extreme Gradient Boosting): Advanced implementation with optimizations:

- **Regularization:** L1 (LASSO) and L2 (Ridge) penalties
- **Handling Missing Values:** Learns best direction for missing
- **Parallel Processing:** Faster training
- **Tree Pruning:** More efficient trees

```
1 import xgboost as xgb
2 import numpy as np
3 from sklearn.datasets import make_classification, make_regression
4 from sklearn.model_selection import train_test_split, GridSearchCV
```

```

5 from sklearn.metrics import accuracy_score, mean_squared_error
6 import pandas as pd
7
8 # Classification example
9 X, y = make_classification(n_samples=1000, n_features=20, random_state=42)
10 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪ random_state=42)
11
12 # Convert to DMatrix (XGBoost's data structure)
13 dtrain = xgb.DMatrix(X_train, label=y_train)
14 dtest = xgb.DMatrix(X_test, label=y_test)
15
16 # Parameters
17 params = {
18     'objective': 'binary:logistic', # Binary classification
19     'eval_metric': 'logloss', # Evaluation metric
20     'eta': 0.1, # Learning rate
21     'max_depth': 6, # Maximum tree depth
22     'min_child_weight': 1, # Minimum sum of instance weight
23     'subsample': 0.8, # Subsample ratio
24     'colsample_bytree': 0.8, # Feature sampling ratio
25     'lambda': 1, # L2 regularization
26     'alpha': 0, # L1 regularization
27     'seed': 42
28 }
29
30 # Train model
31 num_rounds = 100
32 evals = [(dtrain, 'train'), (dtest, 'eval')]
33 bst = xgb.train(
34     params,
35     dtrain,
36     num_rounds,
37     evals=evals,
38     early_stopping_rounds=10,
39     verbose_eval=10
40 )
41
42 # Predictions
43 y_pred_proba = bst.predict(dtest)
44 y_pred = (y_pred_proba > 0.5).astype(int)
45 accuracy = accuracy_score(y_test, y_pred)
46 print(f"Accuracy: {accuracy:.4f}")
47
48 # Feature importance
49 importance = bst.get_score(importance_type='weight')
50 importance_df = pd.DataFrame(

```

```

51     list(importance.items()),
52     columns=['feature', 'importance']
53 ).sort_values('importance', ascending=False)
54 print("\nTop 10 features:")
55 print(importance_df.head(10))
56
57 # Plot feature importance
58 xgb.plot_importance(bst, max_num_features=10)
59 plt.show()
60
61 # Plot tree
62 xgb.plot_tree(bst, num_trees=0) # First tree
63 plt.show()
64
65 # Cross-validation
66 cv_results = xgb.cv(
67     params,
68     dtrain,
69     num_boost_round=100,
70     nfold=5,
71     metrics={'error', 'logloss'},
72     early_stopping_rounds=10,
73     seed=42
74 )
75 print(f"\nCV Results:\n{cv_results.tail()}")
76
77 # Scikit-learn API
78 from xgboost import XGBClassifier
79 xgb_clf = XGBClassifier(
80     n_estimators=100,
81     learning_rate=0.1,
82     max_depth=6,
83     objective='binary:logistic',
84     random_state=42
85 )
86 xgb_clf.fit(X_train, y_train)
87 y_pred_sk = xgb_clf.predict(X_test)
88 print(f"\nScikit-learn API Accuracy: {accuracy_score(y_test, y_pred_sk):.4f}")

```

Listing 12: XGBoost Implementation

LightGBM and CatBoost

```

1 import lightgbm as lgb
2 import catboost as cb
3 from sklearn.model_selection import cross_val_score
4

```

```

5 # LightGBM
6 lgb_train = lgb.Dataset(X_train, y_train)
7 lgb_test = lgb.Dataset(X_test, y_test, reference=lgb_train)
8
9 params_lgb = {
10     'objective': 'binary',
11     'metric': 'binary_logloss',
12     'boosting_type': 'gbdt',
13     'num_leaves': 31,
14     'learning_rate': 0.05,
15     'feature_fraction': 0.9,
16     'bagging_fraction': 0.8,
17     'bagging_freq': 5,
18     'verbose': 0
19 }
20
21 gbm = lgb.train(
22     params_lgb,
23     lgb_train,
24     num_boost_round=100,
25     valid_sets=[lgb_test],
26     early_stopping_rounds=10
27 )
28
29 # CatBoost
30 catboost_model = cb.CatBoostClassifier(
31     iterations=100,
32     learning_rate=0.1,
33     depth=6,
34     loss_function='Logloss',
35     verbose=False,
36     random_state=42
37 )
38
39 catboost_model.fit(X_train, y_train)
40 catboost_score = catboost_model.score(X_test, y_test)
41 print(f"CatBoost Accuracy: {catboost_score:.4f}")

```

Listing 13: LightGBM and CatBoost Comparison

3.2.6 Support Vector Machines

The Intuition: Find the best separating boundary between classes. Not just any boundary, but the one with maximum margin - like finding the widest possible street between classes.

SVM: Maximum Margin Separator

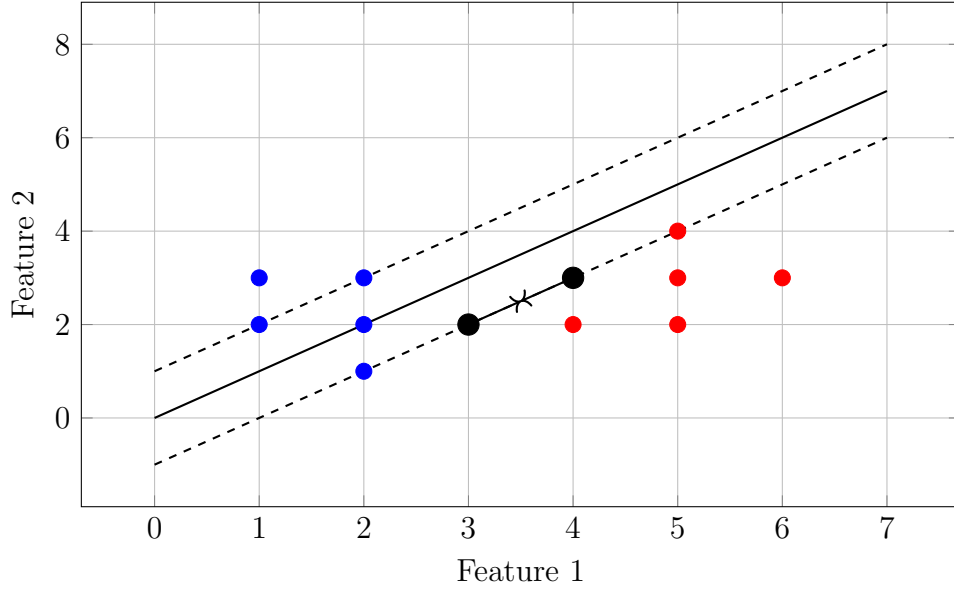


Figure 9: SVM finds hyperplane with maximum margin between classes. Support vectors are points closest to boundary.

Mathematical Formulation: For linearly separable data, find hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ that maximizes margin.

Theorem 3.10 (Hard Margin SVM).

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{s.t.} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad \forall i \end{aligned}$$

*This is a **convex quadratic programming problem**.*

Soft Margin SVM: For non-separable data, allow some misclassification:

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \end{aligned}$$

where C controls trade-off between margin width and misclassification.

The Kernel Trick: For non-linear data, map to higher dimension where it becomes linearly separable:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$$

Common kernels:

- Linear: $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$
- Polynomial: $K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma \mathbf{x}_i^\top \mathbf{x}_j + r)^d$
- RBF: $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$

```

1 from sklearn.svm import SVC
2 from sklearn.datasets import make_moons, make_circles
3 from sklearn.model_selection import train_test_split
4 import matplotlib.pyplot as plt
5 import numpy as np
6
7 # Create non-linear data
8 X, y = make_moons(n_samples=200, noise=0.1, random_state=42)
9
10 # Split data
11 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
12     ↪ random_state=42)
13
14 # Try different kernels
15 kernels = ['linear', 'poly', 'rbf']
16 results = {}
17
18 plt.figure(figsize=(15, 5))
19
20 for idx, kernel in enumerate(kernels):
21     # Create SVM with different kernel
22     svm = SVC(
23         kernel=kernel,
24         C=1.0,
25         gamma='scale',
26         degree=3 if kernel == 'poly' else 3,
27         random_state=42
28     )
29
30     svm.fit(X_train, y_train)
31
32     # Store results
33     results[kernel] = {
34         'model': svm,
35         'train_score': svm.score(X_train, y_train),
36         'test_score': svm.score(X_test, y_test),
37         'n_support_vectors': len(svm.support_vectors_)
38     }
39
40 # Plot decision boundary
41 plt.subplot(1, 3, idx+1)

```

```

42 # Create mesh
43 h = 0.02
44 x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
45 y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
46 xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
47                       np.arange(y_min, y_max, h))
48
49 # Predict on mesh
50 Z = svm.predict(np.c_[xx.ravel(), yy.ravel()])
51 Z = Z.reshape(xx.shape)
52
53 # Plot
54 plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)
55 plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train,
56             cmap=plt.cm.coolwarm, edgecolors='k')
57 plt.scatter(svm.support_vectors_[:, 0], svm.support_vectors_[:, 1],
58             s=100, facecolors='none', edgecolors='k', linewidths=1.5)
59
60 plt.title(f'{kernel.capitalize()} Kernel\n'
61           f'Train: {results[kernel]["train_score"]:.3f}, '
62           f'Test: {results[kernel]["test_score"]:.3f}\n'
63           f'Support Vectors: {results[kernel]["n_support_vectors"]}')
64 plt.xlabel('Feature 1')
65 plt.ylabel('Feature 2')
66
67 plt.tight_layout()
68 plt.show()
69
70 # Print comparison
71 print("Kernel Comparison:")
72 print("=" * 60)
73 for kernel in kernels:
74     print(f"{kernel.upper()} Kernel:")
75     print(f" Training Accuracy: {results[kernel]['train_score']:.4f}")
76     print(f" Test Accuracy: {results[kernel]['test_score']:.4f}")
77     print(f" Number of Support Vectors: {results[kernel]['n_support_vectors']}")
78     print(f" Support Vector Ratio: {results[kernel]['n_support_vectors']/len(
79         ↪ X_train):.2%}")
80     print()

```

Listing 14: SVM Implementation

3.3 Neural Networks

The Intuition: Inspired by the human brain. Simple processing units (neurons) connected together can learn complex patterns through experience.

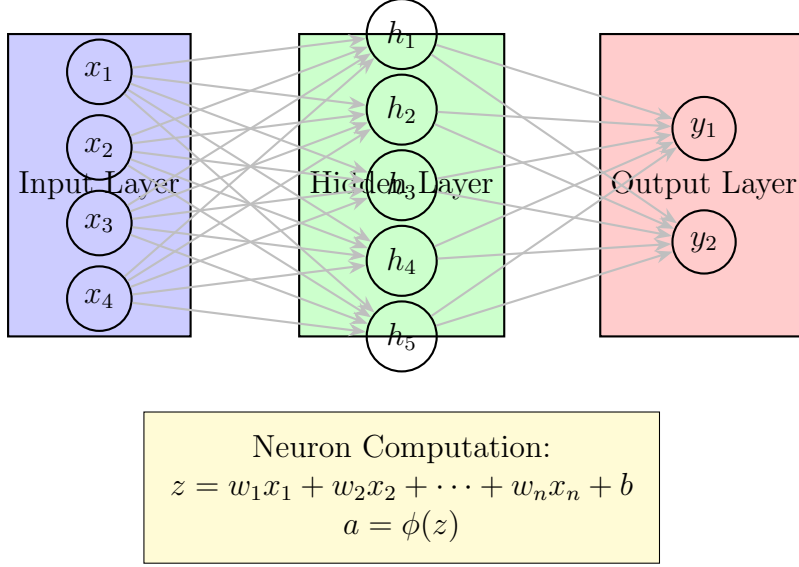


Figure 10: Neural network with input, hidden, and output layers

Basic Building Block: The Neuron Each neuron computes:

$$z = \sum_{i=1}^n w_i x_i + b \quad (\text{weighted sum} + \text{bias})$$

$$a = \phi(z) \quad (\text{activation function})$$

Definition 3.11 (Common Activation Functions).

$$\text{Sigmoid: } \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\text{ReLU: } \text{ReLU}(z) = \max(0, z)$$

$$\text{Leaky ReLU: } \text{LReLU}(z) = \max(\alpha z, z)$$

$$\text{Tanh: } \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\text{Softmax: } \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

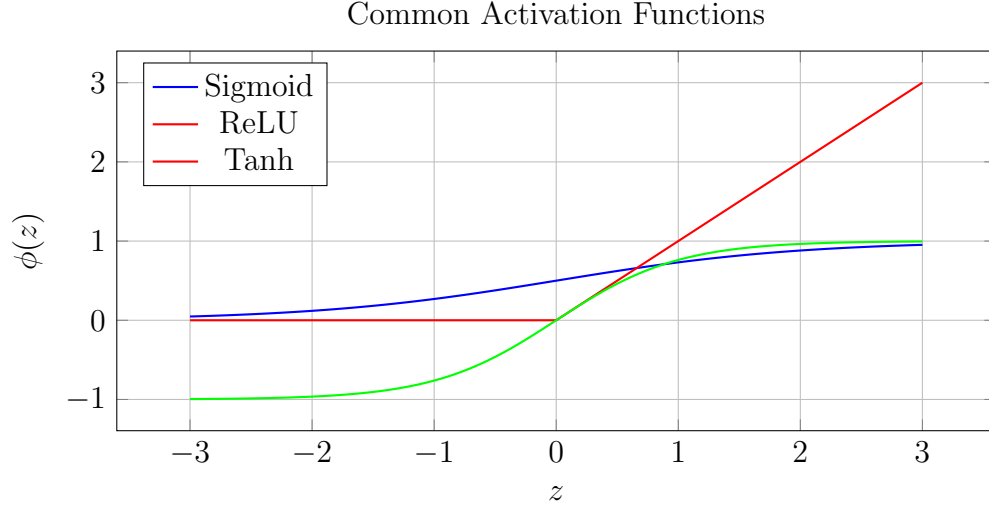


Figure 11: Common activation functions

3.3.1 Backpropagation Algorithm

The algorithm that allows neural networks to learn by propagating errors backward through the network.

Algorithm 8 Backpropagation Algorithm

```

1: procedure BACKPROPAGATION( $X, y, \theta, \eta$ )
2:   # Forward pass: compute activations
3:   for  $l = 1$  to  $L$  do
4:      $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$ 
5:      $a^{(l)} = \phi(z^{(l)})$ 
6:   end for
7:   # Compute output error
8:    $\delta^{(L)} = \nabla_a J \odot \phi'(z^{(L)})$ 
9:   # Backward pass
10:  for  $l = L - 1$  down to 1 do
11:     $\delta^{(l)} = ((W^{(l+1)})^\top \delta^{(l+1)}) \odot \phi'(z^{(l)})$ 
12:  end for
13:  # Update parameters
14:  for  $l = 1$  to  $L$  do
15:     $\nabla_{W^{(l)}} = \delta^{(l)}(a^{(l-1)})^\top$ 
16:     $\nabla_{b^{(l)}} = \delta^{(l)}$ 
17:     $W^{(l)} \leftarrow W^{(l)} - \eta \nabla_{W^{(l)}}$ 
18:     $b^{(l)} \leftarrow b^{(l)} - \eta \nabla_{b^{(l)}}$ 
19:  end for
20: end procedure

```

3.3.2 Python Implementation with PyTorch

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torch.nn.functional as F
5 from torch.utils.data import DataLoader, TensorDataset
6 import numpy as np
7 from sklearn.datasets import make_classification
8 from sklearn.model_selection import train_test_split
9 from sklearn.preprocessing import StandardScaler
10 from sklearn.metrics import accuracy_score
11
12 # Set random seeds for reproducibility
13 torch.manual_seed(42)
14 np.random.seed(42)
15
16 # Generate data
17 X, y = make_classification(
18     n_samples=1000,
19     n_features=20,
20     n_informative=15,
21     n_redundant=5,
22     n_classes=2,
23     random_state=42
24 )
25
26 # Split data
27 X_train, X_test, y_train, y_test = train_test_split(
28     X, y, test_size=0.2, random_state=42
29 )
30
31 # Scale features
32 scaler = StandardScaler()
33 X_train_scaled = scaler.fit_transform(X_train)
34 X_test_scaled = scaler.transform(X_test)
35
36 # Convert to PyTorch tensors
37 X_train_tensor = torch.FloatTensor(X_train_scaled)
38 y_train_tensor = torch.LongTensor(y_train)
39 X_test_tensor = torch.FloatTensor(X_test_scaled)
40 y_test_tensor = torch.LongTensor(y_test)
41
42 # Create datasets and dataloaders
43 train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
44 test_dataset = TensorDataset(X_test_tensor, y_test_tensor)
45
46 train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
47 test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

```

```

48
49 # Define neural network architecture
50 class NeuralNetwork(nn.Module):
51     def __init__(self, input_size, hidden_sizes, output_size, dropout_rate=0.2):
52         super(NeuralNetwork, self).__init__()
53
54         # Create layers
55         layers = []
56         prev_size = input_size
57
58         for i, hidden_size in enumerate(hidden_sizes):
59             layers.append(nn.Linear(prev_size, hidden_size))
60             layers.append(nn.BatchNorm1d(hidden_size))
61             layers.append(nn.ReLU())
62             layers.append(nn.Dropout(dropout_rate))
63             prev_size = hidden_size
64
65         # Output layer
66         layers.append(nn.Linear(prev_size, output_size))
67
68         self.layers = nn.Sequential(*layers)
69
70     def forward(self, x):
71         return self.layers(x)
72
73 # Initialize model
74 input_size = X_train.shape[1]
75 hidden_sizes = [64, 32, 16]
76 output_size = 2
77
78 model = NeuralNetwork(input_size, hidden_sizes, output_size)
79 print(model)
80
81 # Define loss function and optimizer
82 criterion = nn.CrossEntropyLoss()
83 optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)
84 scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.1)
85
86 # Training loop
87 def train_model(model, train_loader, criterion, optimizer, num_epochs=50):
88     model.train()
89     train_losses = []
90     train_accuracies = []
91
92     for epoch in range(num_epochs):
93         running_loss = 0.0
94         correct = 0

```

```

95     total = 0
96
97     for batch_idx, (inputs, targets) in enumerate(train_loader):
98         # Zero gradients
99         optimizer.zero_grad()
100
101         # Forward pass
102         outputs = model(inputs)
103         loss = criterion(outputs, targets)
104
105         # Backward pass
106         loss.backward()
107
108         # Gradient clipping
109         torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
110
111         # Update weights
112         optimizer.step()
113
114         # Statistics
115         running_loss += loss.item()
116         _, predicted = outputs.max(1)
117         total += targets.size(0)
118         correct += predicted.eq(targets).sum().item()
119
120     # Update learning rate
121     scheduler.step()
122
123     # Calculate epoch statistics
124     epoch_loss = running_loss / len(train_loader)
125     epoch_accuracy = 100. * correct / total
126
127     train_losses.append(epoch_loss)
128     train_accuracies.append(epoch_accuracy)
129
130     if (epoch + 1) % 10 == 0:
131         print(f'Epoch [{epoch+1}/{num_epochs}], '
132               f'Loss: {epoch_loss:.4f}, '
133               f'Accuracy: {epoch_accuracy:.2f}%')
134
135     return train_losses, train_accuracies
136
137 # Train the model
138 train_losses, train_accuracies = train_model(
139     model, train_loader, criterion, optimizer, num_epochs=50
140 )
141

```

```

142 # Evaluation
143 def evaluate_model(model, test_loader):
144     model.eval()
145     all_predictions = []
146     all_targets = []
147
148     with torch.no_grad():
149         for inputs, targets in test_loader:
150             outputs = model(inputs)
151             _, predicted = outputs.max(1)
152
153             all_predictions.extend(predicted.cpu().numpy())
154             all_targets.extend(targets.cpu().numpy())
155
156     accuracy = accuracy_score(all_targets, all_predictions)
157     return accuracy, all_predictions, all_targets
158
159 accuracy, predictions, targets = evaluate_model(model, test_loader)
160 print(f"\nTest Accuracy: {accuracy:.4f}")
161
162 # Plot training curves
163 import matplotlib.pyplot as plt
164
165 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
166
167 ax1.plot(train_losses)
168 ax1.set_xlabel('Epoch')
169 ax1.set_ylabel('Loss')
170 ax1.set_title('Training Loss')
171 ax1.grid(True)
172
173 ax2.plot(train_accuracies)
174 ax2.set_xlabel('Epoch')
175 ax2.set_ylabel('Accuracy (%)')
176 ax2.set_title('Training Accuracy')
177 ax2.grid(True)
178
179 plt.tight_layout()
180 plt.show()
181
182 # Save and load model
183 torch.save({
184     'model_state_dict': model.state_dict(),
185     'optimizer_state_dict': optimizer.state_dict(),
186     'scaler': scaler,
187     'input_size': input_size,
188     'hidden_sizes': hidden_sizes,

```

```

189     'output_size': output_size
190 }, 'model.pth')
191
192 # Load model
193 checkpoint = torch.load('model.pth')
194 loaded_model = NeuralNetwork(
195     checkpoint['input_size'],
196     checkpoint['hidden_sizes'],
197     checkpoint['output_size']
198 )
199 loaded_model.load_state_dict(checkpoint['model_state_dict'])

```

Listing 15: Neural Network with PyTorch

3.3.3 Advanced Architectures

```

1 import torch
2 import torch.nn as nn
3 import torchvision
4 import torchvision.transforms as transforms
5
6 # CNN for image classification
7 class CNN(nn.Module):
8     def __init__(self, num_classes=10):
9         super(CNN, self).__init__()
10
11         self.conv_layers = nn.Sequential(
12             # Conv1: 1x28x28 -> 32x28x28
13             nn.Conv2d(1, 32, kernel_size=3, padding=1),
14             nn.BatchNorm2d(32),
15             nn.ReLU(),
16             nn.MaxPool2d(kernel_size=2, stride=2),
17
18             # Conv2: 32x14x14 -> 64x14x14
19             nn.Conv2d(32, 64, kernel_size=3, padding=1),
20             nn.BatchNorm2d(64),
21             nn.ReLU(),
22             nn.MaxPool2d(kernel_size=2, stride=2),
23
24             # Conv3: 64x7x7 -> 128x7x7
25             nn.Conv2d(64, 128, kernel_size=3, padding=1),
26             nn.BatchNorm2d(128),
27             nn.ReLU(),
28             nn.MaxPool2d(kernel_size=2, stride=2),
29         )
30
31         self.fc_layers = nn.Sequential(

```

```

32         nn.Linear(128 * 3 * 3, 256),
33         nn.ReLU(),
34         nn.Dropout(0.5),
35         nn.Linear(256, num_classes)
36     )
37
38     def forward(self, x):
39         x = self.conv_layers(x)
40         x = x.view(x.size(0), -1) # Flatten
41         x = self.fc_layers(x)
42         return x
43
44 # RNN/LSTM for sequence data
45 class LSTMClassifier(nn.Module):
46     def __init__(self, input_size, hidden_size, num_layers, num_classes):
47         super(LSTMClassifier, self).__init__()
48
49         self.hidden_size = hidden_size
50         self.num_layers = num_layers
51
52         self.lstm = nn.LSTM(
53             input_size,
54             hidden_size,
55             num_layers,
56             batch_first=True,
57             dropout=0.2,
58             bidirectional=True
59         )
60
61         self.fc = nn.Linear(hidden_size * 2, num_classes) # *2 for bidirectional
62         self.dropout = nn.Dropout(0.5)
63
64     def forward(self, x):
65         # Initialize hidden and cell states
66         h0 = torch.zeros(self.num_layers * 2, x.size(0), self.hidden_size).to(x.
67             ↪ device)
68         c0 = torch.zeros(self.num_layers * 2, x.size(0), self.hidden_size).to(x.
69             ↪ device)
70
71         # Forward propagate LSTM
72         out, _ = self.lstm(x, (h0, c0))
73
74         # Decode the hidden state of the last time step
75         out = self.dropout(out[:, -1, :])
76         out = self.fc(out)
77
78         return out

```

```

77
78 # Transformer for sequence-to-sequence tasks
79 class TransformerModel(nn.Module):
80     def __init__(self, input_dim, model_dim, num_heads, num_layers, output_dim):
81         super(TransformerModel, self).__init__()
82
83         self.embedding = nn.Linear(input_dim, model_dim)
84         self.pos_encoder = PositionalEncoding(model_dim)
85
86         encoder_layers = nn.TransformerEncoderLayer(
87             model_dim,
88             num_heads,
89             dim_feedforward=2048,
90             dropout=0.1,
91             activation='relu'
92         )
93
94         self.transformer_encoder = nn.TransformerEncoder(encoder_layers,
95             ↪ num_layers)
96         self.fc = nn.Linear(model_dim, output_dim)
97
98     def forward(self, src):
99         src = self.embedding(src)
100         src = self.pos_encoder(src)
101         output = self.transformer_encoder(src)
102         output = output.mean(dim=1) # Mean pooling
103         output = self.fc(output)
104         return output
105
106 class PositionalEncoding(nn.Module):
107     def __init__(self, d_model, max_len=5000):
108         super(PositionalEncoding, self).__init__()
109
110         pe = torch.zeros(max_len, d_model)
111         position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
112         div_term = torch.exp(torch.arange(0, d_model, 2).float() *
113             (-torch.log(torch.tensor(10000.0)) / d_model))
114
115         pe[:, 0::2] = torch.sin(position * div_term)
116         pe[:, 1::2] = torch.cos(position * div_term)
117
118         pe = pe.unsqueeze(0).transpose(0, 1)
119         self.register_buffer('pe', pe)
120
121     def forward(self, x):
122         x = x + self.pe[:x.size(0), :]
123         return x

```

4 Unsupervised Learning

Think of it as: Exploring a new city without a map. You notice patterns (clusters of similar buildings, recurring street layouts) but no one told you what to look for.

When to use it: When you have data but no labels, and you want to discover structure. Examples:

- **Clustering:** Grouping customers with similar buying patterns
- **Dimensionality Reduction:** Finding the most important features in a dataset

The Insight: Even without labels, data often has inherent structure that algorithms can discover.

4.1 Clustering

Purpose: Group similar items together. Examples: customer segmentation, document clustering, image compression.

4.1.1 K-Means Clustering

The Intuition: Divide data into k groups where items in same group are similar and items in different groups are dissimilar. Like organizing books on shelves by topic.

K-Means Clustering: Finding Group Centers

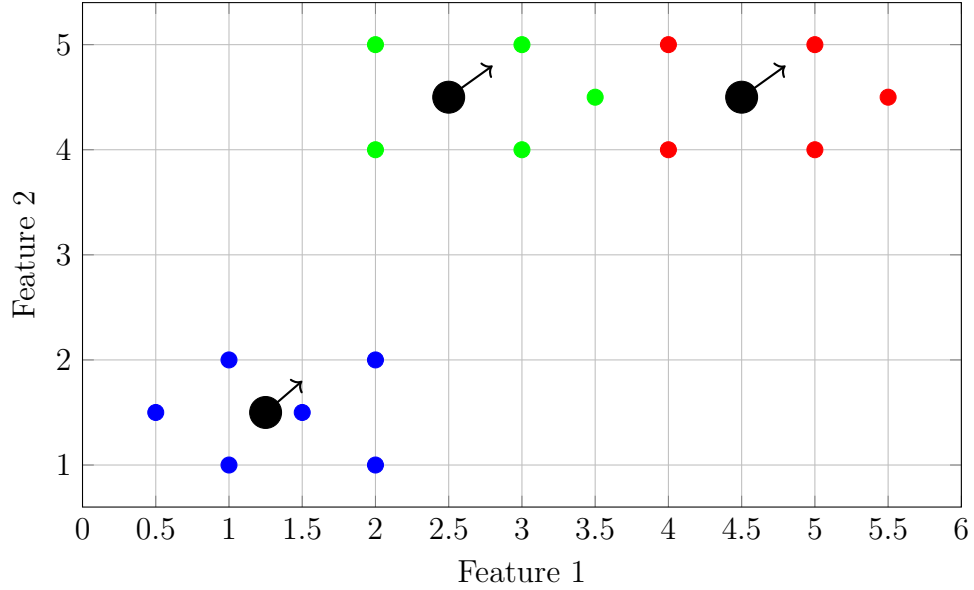


Figure 12: K-means iteratively moves cluster centers (centroids) to minimize within-cluster distance

How K-Means Works:

1. **Initialize:** Randomly choose k points as initial centroids
2. **Assign:** Assign each point to nearest centroid
3. **Update:** Recalculate centroids as mean of assigned points
4. **Repeat:** Until centroids don't change significantly

4.1.2 Mathematical Foundation

Definition 4.1 (K-Means Objective). *Minimize within-cluster sum of squares:*

$$J = \sum_{i=1}^k \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2$$

where $\boldsymbol{\mu}_i$ is centroid of cluster C_i .

Algorithm 9 K-Means Algorithm

```
1: procedure KMEANS( $X, k, \text{max\_iter}$ )
2:   Initialize centroids  $\mu_1, \dots, \mu_k$  (random or k-means++)
3:   for  $\text{iter} = 1$  to  $\text{max\_iter}$  do
4:     # Assignment step
5:     for each point  $\mathbf{x}_i$  do
6:       Assign  $\mathbf{x}_i$  to cluster  $C_j$  with nearest centroid
7:     end for
8:     # Update step
9:     for each cluster  $j = 1$  to  $k$  do
10:       $\mu_j = \frac{1}{|C_j|} \sum_{\mathbf{x} \in C_j} \mathbf{x}$ 
11:    end for
12:    if centroids don't change then
13:      break
14:    end if
15:  end for
16:  return clusters  $C_1, \dots, C_k$ , centroids  $\mu_1, \dots, \mu_k$ 
17: end procedure
```

Definition 4.2 (Elbow Method for Choosing K). *Plot J vs k , choose k at "elbow" point.*

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.cluster import KMeans
4 from sklearn.datasets import make_blobs
5 from sklearn.metrics import silhouette_score, calinski_harabasz_score
6 from sklearn.preprocessing import StandardScaler
7
8 # Generate sample data
9 X, y_true = make_blobs(
10     n_samples=300,
11     centers=4,
12     cluster_std=0.60,
13     random_state=42
14 )
15
16 # Scale data
17 scaler = StandardScaler()
18 X_scaled = scaler.fit_transform(X)
19
20 # Elbow method to find optimal k
21 inertias = []
22 silhouette_scores = []
23 K_range = range(2, 11)
24
25 for k in K_range:
```

```

26     kmeans = KMeans(
27         n_clusters=k,
28         init='k-means++',
29         n_init=10,
30         max_iter=300,
31         random_state=42
32     )
33     kmeans.fit(X_scaled)
34     inertias.append(kmeans.inertia_)
35
36     if k > 1: # Silhouette score requires at least 2 clusters
37         silhouette_scores.append(silhouette_score(X_scaled, kmeans.labels_))
38
39 # Plot elbow method
40 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
41
42 ax1.plot(K_range, inertias, 'bo-')
43 ax1.set_xlabel('Number of clusters (k)')
44 ax1.set_ylabel('Inertia')
45 ax1.set_title('Elbow Method')
46 ax1.grid(True)
47
48 ax2.plot(K_range[1:], silhouette_scores, 'ro-')
49 ax2.set_xlabel('Number of clusters (k)')
50 ax2.set_ylabel('Silhouette Score')
51 ax2.set_title('Silhouette Score Method')
52 ax2.grid(True)
53
54 plt.tight_layout()
55 plt.show()
56
57 # Choose optimal k (elbow point around k=4)
58 optimal_k = 4
59
60 # Fit K-Means with optimal k
61 kmeans = KMeans(
62     n_clusters=optimal_k,
63     init='k-means++',
64     n_init=10,
65     max_iter=300,
66     random_state=42
67 )
68 kmeans.fit(X_scaled)
69 labels = kmeans.labels_
70 centroids = kmeans.cluster_centers_
71
72 # Calculate metrics

```

```

73 silhouette_avg = silhouette_score(X_scaled, labels)
74 calinski_harabasz = calinski_harabasz_score(X_scaled, labels)
75
76 print(f"Silhouette Score: {silhouette_avg:.4f}")
77 print(f"Calinski-Harabasz Score: {calinski_harabasz:.4f}")
78 print(f"Inertia: {kmeans.inertia_:.4f}")
79
80 # Plot clusters
81 plt.figure(figsize=(10, 6))
82 colors = ['red', 'blue', 'green', 'purple', 'orange', 'brown']
83
84 for i in range(optimal_k):
85     plt.scatter(
86         X_scaled[labels == i, 0],
87         X_scaled[labels == i, 1],
88         s=50, c=colors[i], label=f'Cluster {i}'
89     )
90
91 # Plot centroids
92 plt.scatter(
93     centroids[:, 0],
94     centroids[:, 1],
95     s=200, marker='*', c='yellow', edgecolor='black',
96     label='Centroids'
97 )
98
99 plt.xlabel('Feature 1')
100 plt.ylabel('Feature 2')
101 plt.title(f'K-Means Clustering (k={optimal_k})')
102 plt.legend()
103 plt.grid(True)
104 plt.show()
105
106 # Additional analyses
107 print("\nCluster sizes:")
108 unique, counts = np.unique(labels, return_counts=True)
109 for cluster, count in zip(unique, counts):
110     print(f" Cluster {cluster}: {count} points")
111
112 print("\nCluster centroids (scaled):")
113 for i, centroid in enumerate(centroids):
114     print(f" Cluster {i}: {centroid}")
115
116 # Inverse transform to original scale
117 centroids_original = scaler.inverse_transform(centroids)
118 print("\nCluster centroids (original scale):")
119 for i, centroid in enumerate(centroids_original):

```

```
print(f" Cluster {i}: {centroid}")
```

Listing 17: K-Means Clustering

4.1.3 Hierarchical Clustering

The Intuition: Build a tree of clusters. Start with each point as its own cluster, then repeatedly merge closest clusters until all points are in one cluster.

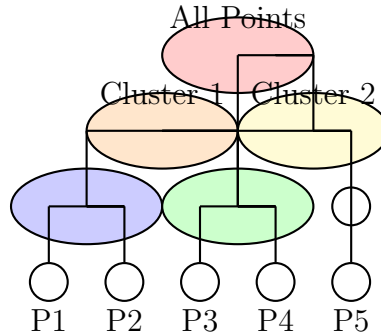


Figure 13: Hierarchical clustering builds tree of clusters (dendrogram)

Types of Hierarchical Clustering:

- **Agglomerative:** Bottom-up, start with individual points
- **Divisive:** Top-down, start with all points in one cluster

Linkage Criteria: How to measure distance between clusters?

- **Single:** Minimum distance between points in clusters
- **Complete:** Maximum distance between points in clusters
- **Average:** Average distance between all pairs
- **Ward:** Minimizes variance when merging

Mathematical Foundation

Definition 4.3 (Linkage Criteria). • **Single:** $d(C_i, C_j) = \min_{x \in C_i, y \in C_j} d(x, y)$

- **Complete:** $d(C_i, C_j) = \max_{x \in C_i, y \in C_j} d(x, y)$
- **Average:** $d(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{x \in C_i} \sum_{y \in C_j} d(x, y)$
- **Ward:** *Minimizes variance when merging clusters*

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
4 from scipy.spatial.distance import pdist
5 from sklearn.datasets import make_blobs
6 from sklearn.preprocessing import StandardScaler
7
8 # Generate sample data
9 X, _ = make_blobs(n_samples=150, centers=3, random_state=42)
10 scaler = StandardScaler()
11 X_scaled = scaler.fit_transform(X)
12
13 # Compute linkage matrix
14 linkage_matrix = linkage(
15     X_scaled,
16     method='ward', # 'single', 'complete', 'average', 'ward'
17     metric='euclidean'
18 )
19
20 # Plot dendrogram
21 plt.figure(figsize=(12, 6))
22 dendrogram(
23     linkage_matrix,
24     truncate_mode='lastp', # Show last p merged clusters
25     p=12,
26     show_leaf_counts=True,
27     leaf_rotation=90,
28     leaf_font_size=12,
29     show_contracted=True
30 )
31 plt.title('Hierarchical Clustering Dendrogram')
32 plt.xlabel('Sample index or (cluster size)')
33 plt.ylabel('Distance')
34 plt.grid(True, alpha=0.3)
35 plt.show()
36
37 # Determine optimal number of clusters using dendrogram
38 # Cut dendrogram at certain distance
39 max_d = 5 # Distance threshold
40 clusters = fcluster(linkage_matrix, max_d, criterion='distance')
41
42 print(f"Number of clusters at distance {max_d}: {len(np.unique(clusters))}")
43
44 # Alternative: cut to get k clusters
45 k = 3
46 clusters_k = fcluster(linkage_matrix, k, criterion='maxclust')
47

```

```

48 # Plot clusters
49 plt.figure(figsize=(10, 6))
50 colors = ['red', 'blue', 'green', 'purple', 'orange']
51
52 for i in range(1, k+1):
53     plt.scatter(
54         X_scaled[clusters_k == i, 0],
55         X_scaled[clusters_k == i, 1],
56         s=50, c=colors[i-1], label=f'Cluster {i}'
57     )
58
59 plt.title(f'Hierarchical Clustering (k={k})')
60 plt.xlabel('Feature 1')
61 plt.ylabel('Feature 2')
62 plt.legend()
63 plt.grid(True)
64 plt.show()
65
66 # Compare different linkage methods
67 methods = ['single', 'complete', 'average', 'ward']
68 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
69
70 for idx, method in enumerate(methods):
71     row, col = divmod(idx, 2)
72     ax = axes[row, col]
73
74     # Compute linkage
75     Z = linkage(X_scaled, method=method)
76
77     # Plot dendrogram
78     dendrogram(Z, ax=ax, truncate_mode='lastp', p=10)
79     ax.set_title(f'Linkage: {method}')
80     ax.set_xlabel('')
81     ax.set_ylabel('Distance')
82
83 plt.tight_layout()
84 plt.show()
85
86 # Cophenetic correlation coefficient (measures how well dendrogram preserves
87     ↪ distances)
88 from scipy.cluster.hierarchy import cophenet
89 from scipy.spatial.distance import squareform
90
91 # Calculate cophenetic correlation for each method
92 for method in methods:
93     Z = linkage(X_scaled, method=method)
94     c, coph_dists = cophenet(Z, pdist(X_scaled))

```

```
print(f"Cophenetic correlation for {method}: {c:.4f}")
```

Listing 18: Hierarchical Clustering

4.1.4 DBSCAN (Density-Based Spatial Clustering)

The Intuition: Find dense regions separated by sparse regions. Like finding groups of people in a park - dense clusters with empty space between them.

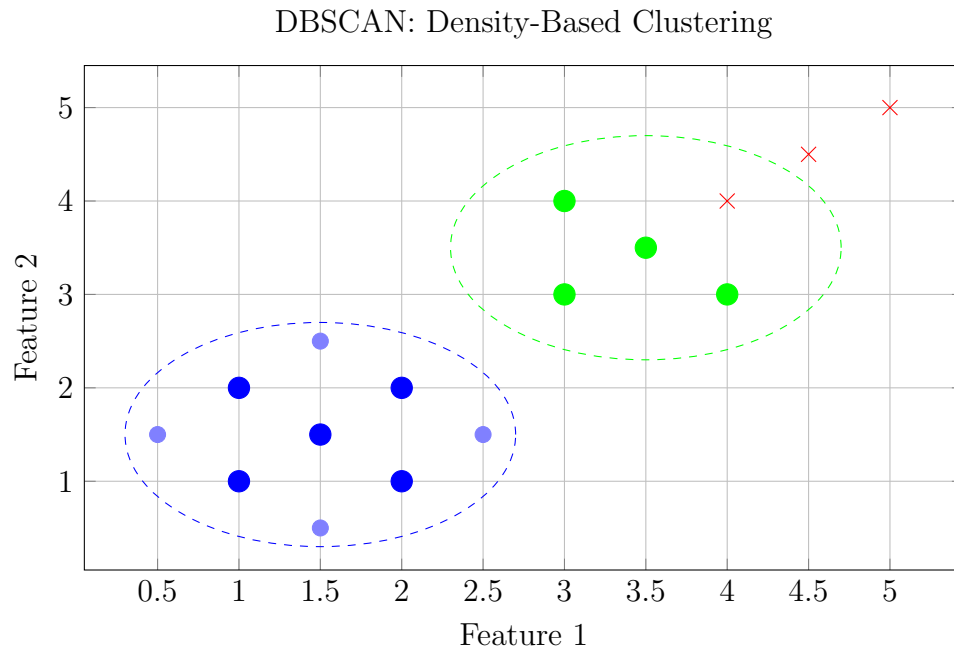


Figure 14: DBSCAN identifies dense regions. Points in dense areas become clusters, isolated points become noise.

DBSCAN Concepts:

- **Core point:** Has at least `min_samples` within distance ϵ
- **Border point:** Within ϵ of core point but not core itself
- **Noise point:** Neither core nor border
- **Density-connected:** Points connected through chain of core points

Advantages:

- Doesn't require specifying number of clusters
- Can find arbitrarily shaped clusters
- Identifies noise/outliers

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.cluster import DBSCAN
4 from sklearn.datasets import make_moons, make_circles
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.neighbors import NearestNeighbors
7 from sklearn.metrics import silhouette_score
8
9 # Generate non-linear data
10 X1, _ = make_moons(n_samples=300, noise=0.05, random_state=42)
11 X2, _ = make_circles(n_samples=300, noise=0.05, factor=0.5, random_state=42)
12 X = np.vstack([X1, X2])
13
14 scaler = StandardScaler()
15 X_scaled = scaler.fit_transform(X)
16
17 # Method 1: Knee/Elbow method to find optimal eps
18 def find_optimal_eps(X, min_samples):
19     """
20     Find optimal epsilon using knee method.
21     Plot k-distance graph and find knee point.
22     """
23     neigh = NearestNeighbors(n_neighbors=min_samples)
24     neigh.fit(X)
25     distances, indices = neigh.kneighbors(X)
26
27     # Sort distances
28     distances = np.sort(distances[:, -1])
29
30     # Find knee point (maximum curvature)
31     # Simplified approach: look for point where slope changes significantly
32     # More sophisticated: use Kneedle algorithm
33
34     return distances
35
36 min_samples = 5
37 distances = find_optimal_eps(X_scaled, min_samples)
38
39 # Plot k-distance graph
40 plt.figure(figsize=(10, 6))
41 plt.plot(range(len(distances)), distances)
42 plt.xlabel('Points sorted by distance')
43 plt.ylabel(f'{min_samples}-th nearest neighbor distance')
44 plt.title('k-Distance Graph for DBSCAN')
45 plt.grid(True)
46
47 # Suggest eps (using percentile)

```

```

48 eps_suggestion = np.percentile(distances, 95)
49 plt.axhline(y=eps_suggestion, color='r', linestyle='--',
50             label=f'Suggested eps: {eps_suggestion:.3f}')
51 plt.legend()
52 plt.show()
53
54 print(f"Suggested eps: {eps_suggestion:.3f}")
55
56 # Apply DBSCAN
57 dbscan = DBSCAN(
58     eps=eps_suggestion,
59     min_samples=min_samples,
60     metric='euclidean',
61     algorithm='auto',
62     leaf_size=30,
63     n_jobs=-1
64 )
65
66 labels = dbscan.fit_predict(X_scaled)
67
68 # Analyze results
69 n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
70 n_noise = list(labels).count(-1)
71
72 print(f"Number of clusters: {n_clusters}")
73 print(f"Number of noise points: {n_noise}")
74 print(f"Percentage of noise: {100 * n_noise / len(X):.2f}%")
75
76 if n_clusters > 1:
77     # Silhouette score (excluding noise points)
78     mask = labels != -1
79     if sum(mask) > 1:
80         score = silhouette_score(X_scaled[mask], labels[mask])
81         print(f"Silhouette Score (excluding noise): {score:.4f}")
82
83 # Visualize clusters
84 plt.figure(figsize=(12, 8))
85
86 # Create colors for clusters
87 unique_labels = set(labels)
88 colors = [plt.cm.Spectral(each) for each in np.linspace(0, 1, len(unique_labels))
89           ↪ ]
90
91 for k, col in zip(unique_labels, colors):
92     if k == -1:
93         # Black used for noise
94         col = [0, 0, 0, 1]

```

```

94         label = 'Noise'
95     else:
96         label = f'Cluster {k}'
97
98     class_member_mask = (labels == k)
99
100     xy = X_scaled[class_member_mask]
101     plt.scatter(
102         xy[:, 0], xy[:, 1],
103         s=50, c=[col], edgecolors='k', alpha=0.7,
104         label=label
105     )
106
107 plt.title(f'DBSCAN Clustering\nEstimated number of clusters: {n_clusters}')
108 plt.xlabel('Feature 1')
109 plt.ylabel('Feature 2')
110 plt.legend()
111 plt.grid(True, alpha=0.3)
112 plt.show()
113
114 # Grid search for optimal parameters
115 from sklearn.model_selection import ParameterGrid
116
117 param_grid = {
118     'eps': [0.1, 0.2, 0.3, 0.4, 0.5],
119     'min_samples': [3, 5, 7, 10]
120 }
121
122 best_score = -1
123 best_params = None
124 best_labels = None
125
126 for params in ParameterGrid(param_grid):
127     dbscan = DBSCAN(**params)
128     labels = dbscan.fit_predict(X_scaled)
129
130     n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
131
132     if n_clusters > 1:
133         mask = labels != -1
134         if sum(mask) > 1:
135             score = silhouette_score(X_scaled[mask], labels[mask])
136
137             if score > best_score:
138                 best_score = score
139                 best_params = params
140                 best_labels = labels

```

```

141
142 print(f"\nBest parameters: {best_params}")
143 print(f"Best silhouette score: {best_score:.4f}")

```

Listing 19: DBSCAN Clustering

4.2 Dimensionality Reduction

Purpose: Reduce number of features while preserving important information. Examples: data visualization, noise reduction, speeding up algorithms.

4.2.1 Principal Component Analysis (PCA)

The Intuition: Find new axes (principal components) that capture most variance in data. Like finding the best angles to view a 3D object to see its true shape.

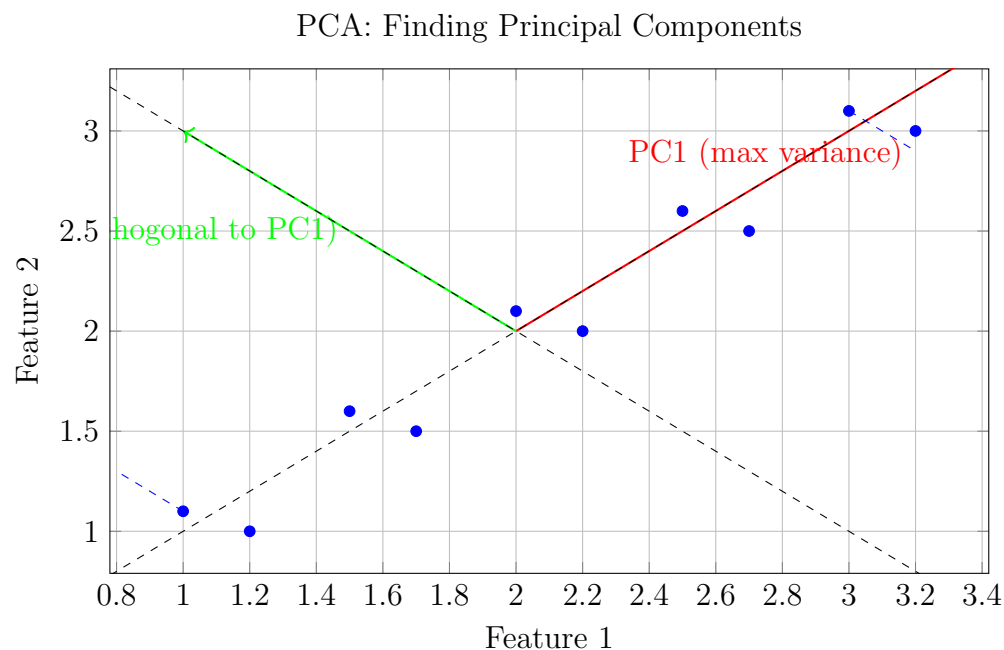


Figure 15: PCA finds directions of maximum variance. PC1 captures most information, PC2 is orthogonal to PC1.

How PCA Works:

1. Center data (subtract mean)
2. Compute covariance matrix
3. Find eigenvectors (principal components) and eigenvalues (variance explained)
4. Sort by eigenvalues (descending)

5. Project data onto top k eigenvectors

Theorem 4.4 (PCA Mathematics). 1. Center: $\mathbf{X}_c = \mathbf{X} - \bar{\mathbf{X}}$

2. Covariance: $\mathbf{S} = \frac{1}{n-1} \mathbf{X}_c^\top \mathbf{X}_c$

3. Eigen decomposition: $\mathbf{S} \mathbf{v}_i = \lambda_i \mathbf{v}_i$

4. Project: $\mathbf{Z} = \mathbf{X}_c \mathbf{V}_k$ where $\mathbf{V}_k$ has top k eigenvectors

Variance Explained:

$$\text{Explained variance ratio} = \frac{\lambda_i}{\sum_{j=1}^p \lambda_j}$$

We typically keep enough components to explain e.g., 95% of variance.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.decomposition import PCA, KernelPCA
4 from sklearn.datasets import load_iris, make_classification
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.model_selection import train_test_split
7 from sklearn.linear_model import LogisticRegression
8 from sklearn.metrics import accuracy_score
9
10 # Load iris dataset
11 iris = load_iris()
12 X = iris.data
13 y = iris.target
14 feature_names = iris.feature_names
15
16 print(f"Original shape: {X.shape}")
17 print(f"Feature names: {feature_names}")
18
19 # Standardize features
20 scaler = StandardScaler()
21 X_scaled = scaler.fit_transform(X)
22
23 # Apply PCA
24 pca = PCA(n_components=None) # Keep all components to see variance
25 X_pca = pca.fit_transform(X_scaled)
26
27 # Explained variance
28 explained_variance = pca.explained_variance_ratio_
29 cumulative_variance = np.cumsum(explained_variance)
30
31 print("\nExplained variance ratio:")
32 for i, (var, cum_var) in enumerate(zip(explained_variance, cumulative_variance),
    ↪ 1):
```

```

33     print(f"PC{i}: {var:.4f} ({cum_var:.4f} cumulative)")
34
35 # Scree plot
36 plt.figure(figsize=(10, 6))
37 plt.subplot(1, 2, 1)
38 plt.bar(range(1, len(explained_variance) + 1), explained_variance, alpha=0.5)
39 plt.plot(range(1, len(explained_variance) + 1), explained_variance, 'ro-')
40 plt.xlabel('Principal Component')
41 plt.ylabel('Explained Variance Ratio')
42 plt.title('Scree Plot')
43 plt.grid(True)
44
45 plt.subplot(1, 2, 2)
46 plt.plot(range(1, len(cumulative_variance) + 1), cumulative_variance, 'bo-')
47 plt.axhline(y=0.95, color='r', linestyle='--', label='95% variance')
48 plt.xlabel('Number of Components')
49 plt.ylabel('Cumulative Explained Variance')
50 plt.title('Cumulative Explained Variance')
51 plt.legend()
52 plt.grid(True)
53
54 plt.tight_layout()
55 plt.show()
56
57 # Choose number of components to explain 95% variance
58 n_components_95 = np.argmax(cumulative_variance >= 0.95) + 1
59 print(f"\nNumber of components for 95% variance: {n_components_95}")
60
61 # Apply PCA with selected number of components
62 pca_selected = PCA(n_components=n_components_95)
63 X_pca_selected = pca_selected.fit_transform(X_scaled)
64 print(f"Reduced shape: {X_pca_selected.shape}")
65
66 # Visualize first two components
67 plt.figure(figsize=(10, 8))
68 scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='viridis', alpha=0.8)
69 plt.xlabel(f'PC1 ({explained_variance[0]:.2%} variance)')
70 plt.ylabel(f'PC2 ({explained_variance[1]:.2%} variance)')
71 plt.title('PCA: First Two Principal Components')
72 plt.colorbar(scatter, label='Class')
73 plt.grid(True, alpha=0.3)
74
75 # Add loadings (feature contributions)
76 loadings = pca.components_.T * np.sqrt(pca.explained_variance_)
77
78 for i, feature in enumerate(feature_names):
79     plt.arrow(0, 0, loadings[i, 0] * 3, loadings[i, 1] * 3,

```

```

80         color='r', alpha=0.5, head_width=0.05)
81     plt.text(loadings[i, 0] * 3.2, loadings[i, 1] * 3.2,
82             feature, color='r', ha='center', va='center')
83
84 plt.axhline(y=0, color='k', linestyle='--', alpha=0.3)
85 plt.axvline(x=0, color='k', linestyle='--', alpha=0.3)
86 plt.show()
87
88 # PCA for classification
89 # Split data
90 X_train, X_test, y_train, y_test = train_test_split(
91     X_scaled, y, test_size=0.2, random_state=42
92 )
93
94 # Train classifier on original features
95 clf_original = LogisticRegression(random_state=42)
96 clf_original.fit(X_train, y_train)
97 y_pred_original = clf_original.predict(X_test)
98 acc_original = accuracy_score(y_test, y_pred_original)
99
100 # Train classifier on PCA features
101 pca_clf = PCA(n_components=2)
102 X_train_pca = pca_clf.fit_transform(X_train)
103 X_test_pca = pca_clf.transform(X_test)
104
105 clf_pca = LogisticRegression(random_state=42)
106 clf_pca.fit(X_train_pca, y_train)
107 y_pred_pca = clf_pca.predict(X_test_pca)
108 acc_pca = accuracy_score(y_test, y_pred_pca)
109
110 print(f"\nClassification Accuracy:")
111 print(f"Original features ({X_train.shape[1]}): {acc_original:.4f}")
112 print(f"PCA features ({X_train_pca.shape[1]}): {acc_pca:.4f}")
113
114 # Kernel PCA for non-linear data
115 from sklearn.datasets import make_circles
116
117 X_circles, y_circles = make_circles(n_samples=400, noise=0.05, random_state=42)
118 X_circles_scaled = StandardScaler().fit_transform(X_circles)
119
120 # Apply different kernel PCAs
121 kernels = ['linear', 'poly', 'rbf', 'cosine']
122 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
123
124 for idx, kernel in enumerate(kernels):
125     row, col = divmod(idx, 2)
126     ax = axes[row, col]

```

```

127     kpca = KernelPCA(n_components=2, kernel=kernel, gamma=10,
128         ↪ fit_inverse_transform=True)
129     X_kpca = kpca.fit_transform(X_circles_scaled)
130
131     ax.scatter(X_kpca[:, 0], X_kpca[:, 1], c=y_circles, cmap='viridis', alpha=0.8)
132     ax.set_title(f'Kernel PCA: {kernel}')
133     ax.set_xlabel('PC1')
134     ax.set_ylabel('PC2')
135     ax.grid(True, alpha=0.3)
136
137 plt.tight_layout()
138 plt.show()
139
140 # PCA for feature extraction
141 print("\nPCA Components (Loadings):")
142 components_df = pd.DataFrame(
143     pca.components_,
144     columns=feature_names,
145     index=[f'PC{i+1}' for i in range(pca.n_components_)])
146 )
147 print(components_df.round(3))
148
149 # Heatmap of loadings
150 import seaborn as sns
151 plt.figure(figsize=(10, 8))
152 sns.heatmap(components_df, annot=True, cmap='RdBu_r', center=0)
153 plt.title('PCA Component Loadings')
154 plt.tight_layout()
155 plt.show()

```

Listing 20: PCA for Dimensionality Reduction

5 Model Evaluation and Validation

5.1 Cross-Validation

Why Cross-Validation? To get reliable estimate of model performance and avoid overfitting to a particular train-test split.

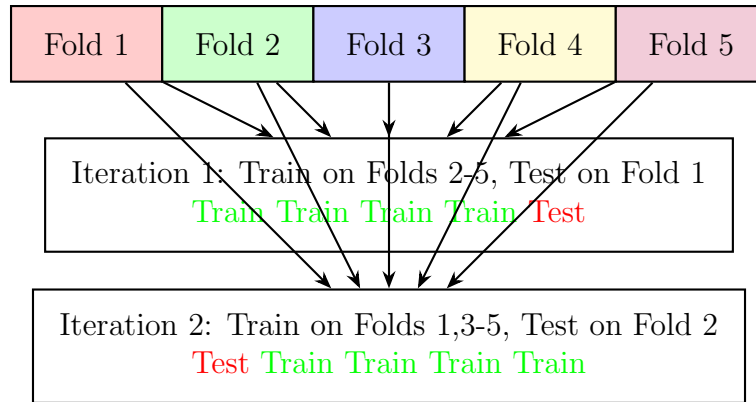


Figure 16: 5-fold cross-validation: Each fold serves as test set once

Types of Cross-Validation:

- **K-Fold:** Split data into k folds
- **Stratified K-Fold:** Preserve class distribution in each fold
- **Leave-One-Out:** Each sample as test set once ($k = n$)
- **Time Series Split:** Respect temporal order

```

1 from sklearn.model_selection import cross_val_score, cross_validate,
   ↪ StratifiedKFold
2 from sklearn.ensemble import RandomForestClassifier
3 from sklearn.datasets import make_classification
4 import numpy as np
5
6 # Generate data
7 X, y = make_classification(n_samples=1000, n_features=20, n_classes=2,
   ↪ random_state=42)
8
9 # Create model
10 model = RandomForestClassifier(n_estimators=100, random_state=42)
11
12 # Simple cross-validation
13 cv_scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
14 print(f"Simple 5-fold CV Accuracy: {cv_scores.mean():.4f} (+/- {cv_scores.std() *
   ↪ 2:.4f})")
15
16 # Stratified k-fold (for imbalanced data)
17 stratified_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
18 stratified_scores = cross_val_score(model, X, y, cv=stratified_cv, scoring='
   ↪ accuracy')
19 print(f"Stratified 5-fold CV Accuracy: {stratified_scores.mean():.4f} (+/- {
   ↪ stratified_scores.std() * 2:.4f})")

```

```

20
21 # Multiple metrics
22 scoring = ['accuracy', 'precision', 'recall', 'f1']
23 cv_results = cross_validate(model, X, y, cv=5, scoring=scoring)
24
25 print("\nDetailed CV Results:")
26 for metric in scoring:
27     scores = cv_results[f'test_{metric}']
28     print(f"{metric}: {scores.mean():.4f} (+/- {scores.std() * 2:.4f})")
29
30 # Leave-One-Out (LOO) - computationally expensive
31 from sklearn.model_selection import LeaveOneOut
32 if len(X) < 100: # LOO is expensive, only for small datasets
33     loo = LeaveOneOut()
34     loo_scores = cross_val_score(model, X, y, cv=loo, scoring='accuracy')
35     print(f"\nLOO CV Accuracy: {loo_scores.mean():.4f}")
36 else:
37     print(f"\nLOO would require {len(X)} iterations - too expensive!")
38
39 # Time series split
40 from sklearn.model_selection import TimeSeriesSplit
41 tscv = TimeSeriesSplit(n_splits=5)
42
43 # For demonstration, create time series data
44 X_ts = np.random.randn(200, 5)
45 y_ts = np.random.randint(0, 2, 200)
46
47 ts_scores = cross_val_score(model, X_ts, y_ts, cv=tscv, scoring='accuracy')
48 print(f"\nTime Series CV Accuracy: {ts_scores.mean():.4f} (+/- {ts_scores.std() *
    ↪ 2:.4f})")

```

Listing 21: Cross-Validation Implementation

5.2 Evaluation Metrics

5.2.1 Classification Metrics

Definition 5.1 (Confusion Matrix).

$$\begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix}$$

Definition 5.2 (Classification Metrics).

$$\begin{aligned}
\text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN} \\
\text{Precision} &= \frac{TP}{TP + FP} \\
\text{Recall/Sensitivity} &= \frac{TP}{TP + FN} \\
\text{Specificity} &= \frac{TN}{TN + FP} \\
\text{F1-Score} &= 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \\
\text{MCC} &= \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}
\end{aligned}$$

Definition 5.3 (ROC and AUC). *ROC curve plots TPR vs FPR. AUC = Area under ROC curve (higher is better).*

5.2.2 Regression Metrics

$$\begin{aligned}
\text{MSE} &= \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \\
\text{RMSE} &= \sqrt{\text{MSE}} \\
\text{MAE} &= \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \\
R^2 &= 1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}} = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2} \\
\text{Adjusted } R^2 &= 1 - \frac{(1 - R^2)(n - 1)}{n - p - 1}
\end{aligned}$$

5.2.3 Clustering Metrics

$$\begin{aligned}
\text{Silhouette Score} &= \frac{b - a}{\max(a, b)} \\
\text{Calinski-Harabasz} &= \frac{\text{SS}_B}{\text{SS}_W} \times \frac{n - k}{k - 1} \\
\text{Davies-Bouldin} &= \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{s_i + s_j}{d(c_i, c_j)} \right)
\end{aligned}$$

5.3 Python Implementation for Evaluation

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.model_selection import cross_val_score, cross_validate
5 from sklearn.metrics import (accuracy_score, precision_score, recall_score,
6                             f1_score, roc_auc_score, confusion_matrix,
7                             mean_squared_error, mean_absolute_error, r2_score,
8                             classification_report, roc_curve, auc,
9                             precision_recall_curve, average_precision_score)
10 from sklearn.linear_model import LogisticRegression
11 from sklearn.ensemble import RandomForestClassifier
12 from sklearn.datasets import make_classification
13 from sklearn.model_selection import train_test_split, StratifiedKFold
14
15 # Generate data
16 X, y = make_classification(
17     n_samples=1000,
18     n_features=20,
19     n_informative=10,
20     n_classes=2,
21     weights=[0.7, 0.3], # Imbalanced
22     random_state=42
23 )
24
25 X_train, X_test, y_train, y_test = train_test_split(
26     X, y, test_size=0.2, random_state=42, stratify=y
27 )
28
29 # Train models
30 models = {
31     'Logistic Regression': LogisticRegression(random_state=42),
32     'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42)
33 }
34
35 # Comprehensive evaluation function
36 def evaluate_model(model, X_train, X_test, y_train, y_test, model_name):
37     """
38     Comprehensive evaluation of a classification model.
39     """
40     # Train model
41     model.fit(X_train, y_train)
42
43     # Predictions
44     y_pred = model.predict(X_test)
45     y_pred_proba = model.predict_proba(X_test)[: , 1] if hasattr(model, '
46         ↪ predict_proba') else None

```

```

47 # Calculate metrics
48 results = {
49     'Accuracy': accuracy_score(y_test, y_pred),
50     'Precision': precision_score(y_test, y_pred, average='weighted'),
51     'Recall': recall_score(y_test, y_pred, average='weighted'),
52     'F1-Score': f1_score(y_test, y_pred, average='weighted')
53 }
54
55 if y_pred_proba is not None:
56     results['ROC-AUC'] = roc_auc_score(y_test, y_pred_proba)
57     results['PR-AUC'] = average_precision_score(y_test, y_pred_proba)
58
59 # Confusion matrix
60 cm = confusion_matrix(y_test, y_pred)
61
62 # Cross-validation scores
63 cv_metrics = {
64     'accuracy': 'accuracy',
65     'precision': 'precision_weighted',
66     'recall': 'recall_weighted',
67     'f1': 'f1_weighted'
68 }
69
70 if y_pred_proba is not None:
71     cv_metrics['roc_auc'] = 'roc_auc'
72
73 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
74 cv_results = cross_validate(
75     model, X_train, y_train,
76     cv=cv,
77     scoring=cv_metrics,
78     n_jobs=-1
79 )
80
81 cv_scores = {}
82 for metric in cv_metrics:
83     scores = cv_results[f'test_{metric}']
84     cv_scores[f'CV_{metric}'] = f"{scores.mean():.4f} (+/- {scores.std() *
85         ↳ 2:.4f})"
86
87 return results, cm, cv_scores, y_pred_proba
88
89 # Evaluate all models
90 all_results = {}
91
92 for model_name, model in models.items():
93     print(f"\n{'='*60}")

```

```

93     print(f"Evaluating {model_name}")
94     print('='*60)
95
96     results, cm, cv_scores, y_pred_proba = evaluate_model(
97         model, X_train, X_test, y_train, y_test, model_name
98     )
99
100     all_results[model_name] = {
101         'results': results,
102         'cm': cm,
103         'cv_scores': cv_scores,
104         'y_pred_proba': y_pred_proba
105     }
106
107     # Print results
108     print("\nPerformance Metrics:")
109     for metric, value in results.items():
110         print(f" {metric}: {value:.4f}")
111
112     print("\nCross-Validation Scores:")
113     for metric, value in cv_scores.items():
114         print(f" {metric}: {value}")
115
116     print("\nConfusion Matrix:")
117     print(cm)
118
119     # Comparative analysis
120     print("\n" + "="*60)
121     print("MODEL COMPARISON")
122     print("="*60)
123
124     comparison_df = pd.DataFrame({
125         model_name: all_results[model_name]['results']
126         for model_name in models.keys()
127     }).T
128
129     print("\nPerformance Comparison:")
130     print(comparison_df.round(4))
131
132     # Visualizations
133     fig, axes = plt.subplots(2, 2, figsize=(15, 12))
134
135     # 1. Confusion matrices
136     for idx, model_name in enumerate(models.keys()):
137         row, col = divmod(idx, 2)
138         ax = axes[row, col]
139

```

```

140 cm = all_results[model_name]['cm']
141 im = ax.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
142 ax.set_title(f'Confusion Matrix: {model_name}')
143
144 # Add text annotations
145 thresh = cm.max() / 2
146 for i in range(cm.shape[0]):
147     for j in range(cm.shape[1]):
148         ax.text(j, i, format(cm[i, j], 'd'),
149                 ha="center", va="center",
150                 color="white" if cm[i, j] > thresh else "black")
151
152 ax.set_ylabel('True label')
153 ax.set_xlabel('Predicted label')
154 plt.colorbar(im, ax=ax)
155
156 # 2. ROC Curves
157 ax = axes[1, 0]
158 for model_name in models.keys():
159     if all_results[model_name]['y_pred_proba'] is not None:
160         fpr, tpr, _ = roc_curve(y_test, all_results[model_name]['y_pred_proba'])
161         roc_auc = auc(fpr, tpr)
162         ax.plot(fpr, tpr, lw=2, label=f'{model_name} (AUC = {roc_auc:.3f})')
163
164 ax.plot([0, 1], [0, 1], 'k--', lw=2)
165 ax.set_xlim([0.0, 1.0])
166 ax.set_ylim([0.0, 1.05])
167 ax.set_xlabel('False Positive Rate')
168 ax.set_ylabel('True Positive Rate')
169 ax.set_title('ROC Curves')
170 ax.legend(loc="lower right")
171 ax.grid(True, alpha=0.3)
172
173 # 3. Precision-Recall Curves
174 ax = axes[1, 1]
175 for model_name in models.keys():
176     if all_results[model_name]['y_pred_proba'] is not None:
177         precision, recall, _ = precision_recall_curve(
178             y_test, all_results[model_name]['y_pred_proba']
179         )
180         avg_precision = average_precision_score(
181             y_test, all_results[model_name]['y_pred_proba']
182         )
183         ax.plot(recall, precision, lw=2,
184                 label=f'{model_name} (AP = {avg_precision:.3f})')
185
186 ax.set_xlim([0.0, 1.0])

```

```

187 ax.set_ylim([0.0, 1.05])
188 ax.set_xlabel('Recall')
189 ax.set_ylabel('Precision')
190 ax.set_title('Precision-Recall Curves')
191 ax.legend(loc="lower left")
192 ax.grid(True, alpha=0.3)
193
194 plt.tight_layout()
195 plt.show()
196
197 # Feature importance for tree-based models
198 for model_name, model in models.items():
199     if hasattr(model, 'feature_importances_'):
200         importances = model.feature_importances_
201         indices = np.argsort(importances)[::-1]
202
203         plt.figure(figsize=(10, 6))
204         plt.title(f"Feature Importances: {model_name}")
205         plt.bar(range(len(importances)), importances[indices])
206         plt.xticks(range(len(importances)), indices)
207         plt.xlabel("Feature Index")
208         plt.ylabel("Importance")
209         plt.tight_layout()
210         plt.show()
211
212 # Learning curves
213 from sklearn.model_selection import learning_curve
214
215 def plot_learning_curve(model, X, y, title):
216     """
217     Plot learning curves for a model.
218     """
219     train_sizes, train_scores, test_scores = learning_curve(
220         model, X, y,
221         cv=5,
222         n_jobs=-1,
223         train_sizes=np.linspace(0.1, 1.0, 10),
224         scoring='accuracy'
225     )
226
227     train_mean = np.mean(train_scores, axis=1)
228     train_std = np.std(train_scores, axis=1)
229     test_mean = np.mean(test_scores, axis=1)
230     test_std = np.std(test_scores, axis=1)
231
232     plt.figure(figsize=(10, 6))
233     plt.plot(train_sizes, train_mean, 'o-', color='blue', label='Training score')

```

```

234 plt.plot(train_sizes, test_mean, 'o-', color='green', label='Cross-validation
    ↳ score')
235
236 plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std,
    ↳ alpha=0.1, color='blue')
237 plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std,
    ↳ alpha=0.1, color='green')
238
239 plt.xlabel('Training examples')
240 plt.ylabel('Accuracy')
241 plt.title(f'Learning Curves: {title}')
242 plt.legend(loc='best')
243 plt.grid(True, alpha=0.3)
244 plt.show()
245
246 # Plot learning curves for each model
247 for model_name, model in models.items():
248     plot_learning_curve(model, X_train, y_train, model_name)

```

Listing 22: Comprehensive Model Evaluation

6 Hyperparameter Tuning

subsectionHyperparameter Tuning

What are Hyperparameters? Parameters set before training (not learned from data). Examples: learning rate, tree depth, regularization strength.

6.0.1 Grid Search

Try all combinations of hyperparameter values.

```

1 from sklearn.model_selection import GridSearchCV
2 from sklearn.svm import SVC
3 from sklearn.datasets import load_iris
4 from sklearn.model_selection import train_test_split
5
6 # Load data
7 iris = load_iris()
8 X, y = iris.data, iris.target
9 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↳ random_state=42)
10
11 # Define parameter grid
12 param_grid = {
13     'C': [0.1, 1, 10, 100], # Regularization strength
14     'gamma': ['scale', 'auto', 0.1, 1, 10], # Kernel coefficient
15     'kernel': ['rbf', 'linear', 'poly'] # Kernel type

```

```

16 }
17
18 # Create grid search
19 grid_search = GridSearchCV(
20     SVC(random_state=42),
21     param_grid,
22     cv=5, # 5-fold cross-validation
23     scoring='accuracy',
24     n_jobs=-1, # Use all CPU cores
25     verbose=1
26 )
27
28 # Fit grid search
29 grid_search.fit(X_train, y_train)
30
31 print(f"Best parameters: {grid_search.best_params_}")
32 print(f"Best cross-validation score: {grid_search.best_score_:.4f}")
33 print(f"Test set score: {grid_search.score(X_test, y_test):.4f}")
34
35 # View all results
36 import pandas as pd
37 results_df = pd.DataFrame(grid_search.cv_results_)
38 print("\nTop 5 parameter combinations:")
39 top_results = results_df.sort_values('mean_test_score', ascending=False).head()
40 print(top_results[['params', 'mean_test_score', 'std_test_score']])

```

Listing 23: Grid Search Implementation

6.0.2 Randomized Search

Sample random combinations from parameter distributions.

```

1 from sklearn.model_selection import RandomizedSearchCV
2 from scipy.stats import uniform, randint
3 from sklearn.ensemble import RandomForestClassifier
4
5 # Define parameter distributions
6 param_distributions = {
7     'n_estimators': randint(50, 500), # Random integer between 50 and 500
8     'max_depth': [None, 10, 20, 30, 50],
9     'min_samples_split': randint(2, 20),
10    'min_samples_leaf': randint(1, 10),
11    'max_features': ['sqrt', 'log2', None],
12    'bootstrap': [True, False]
13 }
14
15 # Create randomized search
16 random_search = RandomizedSearchCV(

```

```

17     RandomForestClassifier(random_state=42),
18     param_distributions,
19     n_iter=50, # Number of parameter settings sampled
20     cv=5,
21     scoring='accuracy',
22     n_jobs=-1,
23     random_state=42,
24     verbose=1
25 )
26
27 random_search.fit(X_train, y_train)
28
29 print(f"Best parameters: {random_search.best_params_}")
30 print(f"Best cross-validation score: {random_search.best_score_:.4f}")
31 print(f"Test set score: {random_search.score(X_test, y_test):.4f}")

```

Listing 24: Randomized Search Implementation

6.0.3 Bayesian Optimization

Use probabilistic model to guide search toward promising regions.

```

1 import optuna
2 from sklearn.model_selection import cross_val_score
3
4 def objective(trial):
5     """
6     Define objective function for Optuna optimization.
7     """
8     # Define search space
9     n_estimators = trial.suggest_int('n_estimators', 50, 500)
10    max_depth = trial.suggest_int('max_depth', 3, 30)
11    min_samples_split = trial.suggest_int('min_samples_split', 2, 20)
12    min_samples_leaf = trial.suggest_int('min_samples_leaf', 1, 10)
13    max_features = trial.suggest_categorical('max_features', ['sqrt', 'log2', None
14    ↪ ])
15
16    # Create model
17    model = RandomForestClassifier(
18        n_estimators=n_estimators,
19        max_depth=max_depth,
20        min_samples_split=min_samples_split,
21        min_samples_leaf=min_samples_leaf,
22        max_features=max_features,
23        random_state=42
24    )
25
26    # Evaluate using cross-validation

```

```

26     scores = cross_val_score(model, X_train, y_train,
27                               cv=5, scoring='accuracy', n_jobs=-1)
28
29     return scores.mean()
30
31 # Create study
32 study = optuna.create_study(direction='maximize', sampler=optuna.samplers.
    ↪ TPESampler())
33 study.optimize(objective, n_trials=50, n_jobs=-1)
34
35 # Results
36 print(f"Best trial:")
37 trial = study.best_trial
38 print(f" Value (accuracy): {trial.value:.4f}")
39 print(f" Params: {trial.params}")
40
41 # Train final model with best parameters
42 best_params = trial.params
43 best_model = RandomForestClassifier(**best_params, random_state=42)
44 best_model.fit(X_train, y_train)
45 test_score = best_model.score(X_test, y_test)
46 print(f"\nTest set accuracy with optimized parameters: {test_score:.4f}")
47
48 # Visualize optimization
49 optuna.visualization.plot_optimization_history(study).show()
50 optuna.visualization.plot_param_importances(study).show()

```

Listing 25: Bayesian Optimization with Optuna

7 Advanced Topics and Best Practices

7.1 Handling Imbalanced Data

The Problem: When one class has many more examples than others. Example: Fraud detection (99% legitimate, 1% fraud).

Techniques:

1. Resampling:

- **Oversampling:** Duplicate minority class samples (SMOTE)
- **Undersampling:** Remove majority class samples

2. Class Weights: Give higher penalty for misclassifying minority class

3. Ensemble Methods: Use algorithms designed for imbalance

```

1 from imblearn.over_sampling import SMOTE
2 from imblearn.under_sampling import RandomUnderSampler
3 from imblearn.combine import SMOTETomek
4 from sklearn.metrics import classification_report
5 import numpy as np
6
7 # Create imbalanced data
8 from sklearn.datasets import make_classification
9 X, y = make_classification(
10     n_samples=1000,
11     n_features=20,
12     n_classes=2,
13     weights=[0.95, 0.05], # 95% class 0, 5% class 1
14     random_state=42
15 )
16
17 print(f"Original class distribution: {np.bincount(y)}")
18
19 # Different resampling techniques
20 techniques = {
21     'Original': None,
22     'SMOTE': SMOTE(random_state=42),
23     'Random Undersampling': RandomUnderSampler(random_state=42),
24     'SMOTE + Tomek': SMOTETomek(random_state=42)
25 }
26
27 from sklearn.ensemble import RandomForestClassifier
28 from sklearn.model_selection import train_test_split
29
30 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
31     ↪ random_state=42)
32
33 for name, sampler in techniques.items():
34     print(f"\n{'='*50}")
35     print(f"Technique: {name}")
36     print('='*50)
37
38     if sampler:
39         X_resampled, y_resampled = sampler.fit_resample(X_train, y_train)
40         print(f"Resampled class distribution: {np.bincount(y_resampled)}")
41     else:
42         X_resampled, y_resampled = X_train, y_train
43
44 # Train model
45 model = RandomForestClassifier(random_state=42)
46 model.fit(X_resampled, y_resampled)

```

```

47 # Evaluate
48 y_pred = model.predict(X_test)
49 print("\nClassification Report:")
50 print(classification_report(y_test, y_pred))
51
52 # Focus on minority class (class 1)
53 from sklearn.metrics import recall_score
54 minority_recall = recall_score(y_test, y_pred, pos_label=1)
55 print(f"Recall for minority class (1): {minority_recall:.4f}")

```

Listing 26: Handling Imbalanced Data

7.2 Feature Engineering Best Practices

Why Feature Engineering Matters: "Garbage in, garbage out." Good features can make simple models perform well; bad features can make complex models fail.

Key Techniques:

1. Handling Missing Values:

- Mean/median imputation for numerical
- Mode imputation for categorical
- KNN imputation for relationships
- Create missing indicator

2. Encoding Categorical Variables:

- One-hot encoding for nominal, few categories
- Label encoding for ordinal
- Target encoding for high cardinality

3. Feature Creation:

- Interaction terms (feature1 $\times$ feature2)
- Polynomial features
- Aggregations (mean, sum, count)
- Time-based features

4. Feature Scaling:

- Standardization (mean=0, std=1)
- Min-max scaling to [0, 1]
- Robust scaling (using median, IQR)

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.pipeline import Pipeline
4 from sklearn.compose import ColumnTransformer
5 from sklearn.impute import SimpleImputer
6 from sklearn.preprocessing import (StandardScaler, OneHotEncoder,
7                                   PolynomialFeatures, FunctionTransformer)
8 from sklearn.feature_selection import SelectKBest, f_classif
9
10 # Create sample data with mixed types
11 np.random.seed(42)
12 n_samples = 1000
13
14 data = pd.DataFrame({
15     'age': np.random.randint(18, 70, n_samples),
16     'income': np.random.normal(50000, 15000, n_samples),
17     'education': np.random.choice(['HS', 'College', 'Grad'], n_samples),
18     'city': np.random.choice(['NYC', 'LA', 'Chicago', 'Miami'], n_samples),
19     'purchase_amount': np.random.exponential(100, n_samples),
20     'missing_feature': np.random.choice([1, 2, 3, np.nan], n_samples, p=[0.3, 0.3,
21                                     ↪ 0.3, 0.1])
22 })
23
24 # Create target (for demonstration)
25 data['target'] = (
26     (data['age'] > 40).astype(int) +
27     (data['income'] > 60000).astype(int) +
28     (data['education'] == 'Grad').astype(int) +
29     np.random.randint(0, 2, n_samples)
30 ) > 2
31
32 # Separate features and target
33 X = data.drop('target', axis=1)
34 y = data['target']
35
36 # Define numerical and categorical columns
37 numerical_cols = ['age', 'income', 'purchase_amount', 'missing_feature']
38 categorical_cols = ['education', 'city']
39
40 # Create preprocessing pipelines
41 numerical_pipeline = Pipeline([
42     ('imputer', SimpleImputer(strategy='median')), # Handle missing values
43     ('scaler', StandardScaler()), # Scale features
44     ('polynomial', PolynomialFeatures(degree=2, include_bias=False)) # Add
45     ↪ interactions
46 ])

```

```

46 categorical_pipeline = Pipeline([
47     ('imputer', SimpleImputer(strategy='most_frequent')), # Handle missing
48     ↪ categories
49     ('encoder', OneHotEncoder(handle_unknown='ignore')) # Encode categories
50 ])
51
52 # Combine pipelines
53 preprocessor = ColumnTransformer([
54     ('num', numerical_pipeline, numerical_cols),
55     ('cat', categorical_pipeline, categorical_cols)
56 ])
57
58 # Create full pipeline with feature selection
59 full_pipeline = Pipeline([
60     ('preprocessor', preprocessor),
61     ('feature_selection', SelectKBest(score_func=f_classif, k=20)),
62 ])
63
64 # Apply pipeline
65 X_processed = full_pipeline.fit_transform(X, y)
66
67 print(f"Original shape: {X.shape}")
68 print(f"Processed shape: {X_processed.shape}")
69 print(f"Number of features increased from {X.shape[1]} to {X_processed.shape[1]}
70     ↪ after polynomial features")
71 print(f"Selected {X_processed.shape[1]} best features")
72
73 # Show feature names (for illustration)
74 # Get feature names after preprocessing
75 preprocessor_names = (
76     [f"num_{i}" for i in range(len(numerical_cols) + len(numerical_cols) * (len(
77     ↪ numerical_cols) - 1) // 2)] +
78     [f"cat_{i}" for i in range(len(categorical_cols) * 3)] # 3 unique values in
79     ↪ each categorical
80 )
81
82 print("\nPipeline steps:")
83 for step_name, step in full_pipeline.named_steps.items():
84     print(f"{step_name}: {step}")

```

Listing 27: Comprehensive Feature Engineering Pipeline

7.3 Time Series Forecasting

What is Time Series Data? A sequence of data points indexed in time order. Unlike cross-sectional data where each observation is independent, time series data points are correlated with previous values.

Definition 7.1 (Time Series). *A time series is a set of observations y_t , each recorded at a specific time t . Formally:*

$$\{y_t : t \in T\}$$

where T is an ordered set of time indices.

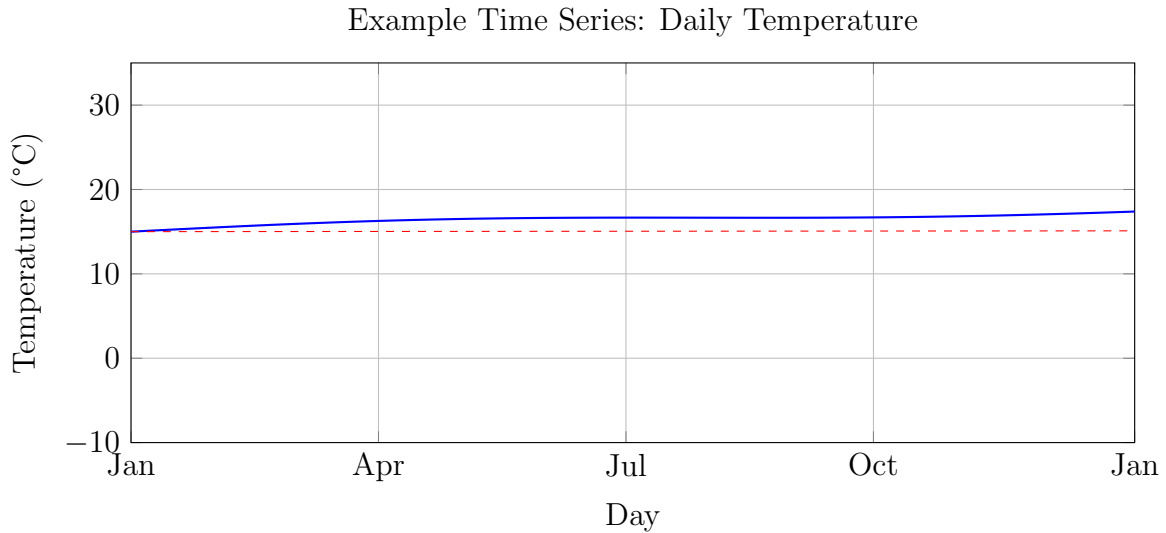


Figure 17: Time series showing seasonal patterns and trend

Key Components of Time Series:

- **Trend:** Long-term upward or downward movement
- **Seasonality:** Regular pattern that repeats at fixed intervals
- **Cycle:** Longer-term fluctuations without fixed period
- **Residual/Noise:** Random, unpredictable component

Time Series Decomposition

Definition 7.2 (Additive Decomposition).

$$y_t = T_t + S_t + R_t$$

where:

- T_t : Trend component
- S_t : Seasonal component
- R_t : Residual component

Definition 7.3 (Multiplicative Decomposition).

$$y_t = T_t \times S_t \times R_t$$

Often used when seasonal variation increases with level.

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from statsmodels.tsa.stattools import adfuller, acf, pacf
5 from statsmodels.tsa.arima.model import ARIMA
6 from statsmodels.tsa.seasonal import seasonal_decompose
7 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
8 from sklearn.metrics import mean_squared_error, mean_absolute_error
9 import warnings
10 warnings.filterwarnings('ignore')
11
12 # Generate synthetic time series data
13 np.random.seed(42)
14 n_periods = 200
15 time = pd.date_range(start='2020-01-01', periods=n_periods, freq='D')
16
17 # Create time series with trend, seasonality, and noise
18 trend = np.linspace(0, 10, n_periods)
19 seasonality = 5 * np.sin(2 * np.pi * np.arange(n_periods) / 30) # Monthly
20     ↪ seasonality
21 noise = np.random.normal(0, 1, n_periods)
22
23 series = trend + seasonality + noise
24 df = pd.DataFrame({'value': series}, index=time)
25
26 print(f"Time series shape: {df.shape}")
27 print(f"Date range: {df.index.min()} to {df.index.max()}")
28
29 # 1. Time Series Visualization
30 plt.figure(figsize=(15, 10))
31
32 # Original series
33 plt.subplot(3, 2, 1)
34 plt.plot(df.index, df['value'])
35 plt.title('Original Time Series')
36 plt.xlabel('Date')
37 plt.ylabel('Value')
38 plt.grid(True, alpha=0.3)
39
40 # Rolling statistics
41 rolling_mean = df['value'].rolling(window=30).mean()
42 rolling_std = df['value'].rolling(window=30).std()
```

```

42 plt.subplot(3, 2, 2)
43 plt.plot(df.index, df['value'], label='Original')
44 plt.plot(df.index, rolling_mean, label='Rolling Mean (30 days)', color='red')
45 plt.plot(df.index, rolling_std, label='Rolling Std (30 days)', color='green')
46 plt.title('Rolling Statistics')
47 plt.xlabel('Date')
48 plt.ylabel('Value')
49 plt.legend()
50 plt.grid(True, alpha=0.3)
51
52
53 # 2. Stationarity Test (Augmented Dickey-Fuller)
54 print("\n" + "="*60)
55 print("STATIONARITY TEST")
56 print("="*60)
57
58 def test_stationarity(timeseries):
59     """
60     Perform Dickey-Fuller test for stationarity.
61     """
62     print('Results of Dickey-Fuller Test:')
63     dfctest = adfuller(timeseries, autolag='AIC')
64     dfoutput = pd.Series(dfctest[0:4],
65                          index=['Test Statistic', 'p-value',
66                                '#Lags Used', 'Number of Observations Used'])
67
68     for key, value in dfctest[4].items():
69         dfoutput[f'Critical Value ({key})'] = value
70
71     print(dfoutput)
72
73     # Interpretation
74     if dfctest[1] <= 0.05:
75         print("\nConclusion: The time series is stationary (reject H0)")
76     else:
77         print("\nConclusion: The time series is non-stationary (fail to reject H0)
78             ↪ ")
79
80 test_stationarity(df['value'])
81
82 # 3. Decomposition
83 print("\n" + "="*60)
84 print("TIME SERIES DECOMPOSITION")
85 print("="*60)
86
87 decomposition = seasonal_decompose(df['value'], model='additive', period=30)

```

```

88 plt.subplot(3, 2, 3)
89 plt.plot(decomposition.trend)
90 plt.title('Trend Component')
91 plt.xlabel('Date')
92 plt.ylabel('Value')
93 plt.grid(True, alpha=0.3)
94
95 plt.subplot(3, 2, 4)
96 plt.plot(decomposition.seasonal)
97 plt.title('Seasonal Component')
98 plt.xlabel('Date')
99 plt.ylabel('Value')
100 plt.grid(True, alpha=0.3)
101
102 plt.subplot(3, 2, 5)
103 plt.plot(decomposition.resid)
104 plt.title('Residual Component')
105 plt.xlabel('Date')
106 plt.ylabel('Value')
107 plt.grid(True, alpha=0.3)
108
109 plt.tight_layout()
110 plt.show()
111
112 # 4. ACF and PACF plots
113 print("\n" + "="*60)
114 print("AUTOCORRELATION ANALYSIS")
115 print("="*60)
116
117 fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))
118 plot_acf(df['value'], lags=40, ax=ax1)
119 ax1.set_title('Autocorrelation Function (ACF)')
120 ax1.grid(True, alpha=0.3)
121
122 plot_pacf(df['value'], lags=40, ax=ax2)
123 ax2.set_title('Partial Autocorrelation Function (PACF)')
124 ax2.grid(True, alpha=0.3)
125
126 plt.tight_layout()
127 plt.show()
128
129 # 5. ARIMA Model
130 print("\n" + "="*60)
131 print("ARIMA MODELING")
132 print("="*60)
133
134 # Split data into train and test

```

```

135 train_size = int(len(df) * 0.8)
136 train, test = df.iloc[:train_size], df.iloc[train_size:]
137
138 print(f"Train size: {len(train)}")
139 print(f"Test size: {len(test)}")
140
141 # Determine ARIMA parameters (p, d, q)
142 # Based on ACF and PACF plots, we can try different combinations
143
144 # Grid search for best ARIMA parameters
145 best_aic = np.inf
146 best_order = None
147 best_model = None
148
149 # Try different combinations (simplified for example)
150 p_values = range(0, 3)
151 d_values = range(0, 2)
152 q_values = range(0, 3)
153
154 for p in p_values:
155     for d in d_values:
156         for q in q_values:
157             try:
158                 model = ARIMA(train['value'], order=(p, d, q))
159                 model_fit = model.fit()
160
161                 if model_fit.aic < best_aic:
162                     best_aic = model_fit.aic
163                     best_order = (p, d, q)
164                     best_model = model_fit
165
166             except:
167                 continue
168
169 print(f"Best ARIMA order: {best_order}")
170 print(f"Best AIC: {best_aic:.2f}")
171
172 # Fit best model
173 model = ARIMA(train['value'], order=best_order)
174 model_fit = model.fit()
175 print(model_fit.summary())
176
177 # Forecast
178 forecast_steps = len(test)
179 forecast = model_fit.forecast(steps=forecast_steps)
180 forecast_index = test.index
181

```

```

182 # Calculate metrics
183 mse = mean_squared_error(test['value'], forecast)
184 rmse = np.sqrt(mse)
185 mae = mean_absolute_error(test['value'], forecast)
186 mape = np.mean(np.abs((test['value'] - forecast) / test['value'])) * 100
187
188 print(f"\nForecast Metrics:")
189 print(f"MSE: {mse:.4f}")
190 print(f"RMSE: {rmse:.4f}")
191 print(f"MAE: {mae:.4f}")
192 print(f"MAPE: {mape:.2f}%")
193
194 # Plot results
195 plt.figure(figsize=(15, 8))
196 plt.plot(train.index, train['value'], label='Training Data', color='blue')
197 plt.plot(test.index, test['value'], label='Actual Test Data', color='green')
198 plt.plot(forecast_index, forecast, label='ARIMA Forecast', color='red', linestyle
    ↪ = '--')
199 plt.fill_between(forecast_index,
200                 forecast - 1.96 * model_fit.resid.std(),
201                 forecast + 1.96 * model_fit.resid.std(),
202                 color='red', alpha=0.2, label='95% Confidence Interval')
203 plt.title(f'ARIMA({best_order[0]},{best_order[1]},{best_order[2]}) Forecast')
204 plt.xlabel('Date')
205 plt.ylabel('Value')
206 plt.legend()
207 plt.grid(True, alpha=0.3)
208 plt.show()
209
210 # 6. Prophet (Facebook's time series forecasting)
211 print("\n" + "="*60)
212 print("PROPHET MODEL")
213 print("="*60)
214
215 from prophet import Prophet
216
217 # Prepare data for Prophet
218 df_prophet = df.reset_index()
219 df_prophet.columns = ['ds', 'y']
220
221 # Split
222 train_prophet = df_prophet.iloc[:train_size]
223 test_prophet = df_prophet.iloc[train_size:]
224
225 # Create and fit Prophet model
226 prophet_model = Prophet(
227     yearly_seasonality=False,

```

```

228     weekly_seasonality=False,
229     daily_seasonality=False,
230     seasonality_mode='additive',
231     changepoint_prior_scale=0.05
232 )
233
234 # Add monthly seasonality
235 prophet_model.add_seasonality(name='monthly', period=30.5, fourier_order=5)
236
237 prophet_model.fit(train_prophet)
238
239 # Make future dataframe
240 future = prophet_model.make_future_dataframe(periods=len(test_prophet))
241 forecast_prophet = prophet_model.predict(future)
242
243 # Extract forecast for test period
244 forecast_test = forecast_prophet.iloc[train_size:][['ds', 'yhat', 'yhat_lower', '
    ↪ yhat_upper']]
245
246 # Calculate metrics
247 mse_prophet = mean_squared_error(test_prophet['y'], forecast_test['yhat'])
248 rmse_prophet = np.sqrt(mse_prophet)
249 mae_prophet = mean_absolute_error(test_prophet['y'], forecast_test['yhat'])
250
251 print(f"\nProphet Forecast Metrics:")
252 print(f"MSE: {mse_prophet:.4f}")
253 print(f"RMSE: {rmse_prophet:.4f}")
254 print(f"MAE: {mae_prophet:.4f}")
255
256 # Plot Prophet results
257 fig1 = prophet_model.plot(forecast_prophet)
258 plt.title('Prophet Forecast')
259 plt.show()
260
261 fig2 = prophet_model.plot_components(forecast_prophet)
262 plt.show()
263
264 # 7. Compare models
265 print("\n" + "="*60)
266 print("MODEL COMPARISON")
267 print("="*60)
268
269 comparison_df = pd.DataFrame({
270     'Model': ['ARIMA', 'Prophet'],
271     'RMSE': [rmse, rmse_prophet],
272     'MAE': [mae, mae_prophet]
273 })

```

```

274
275 print(comparison_df)
276
277 # Plot comparison
278 plt.figure(figsize=(12, 6))
279 plt.plot(test.index, test['value'], label='Actual', color='black', linewidth=2)
280 plt.plot(test.index, forecast, label=f'ARIMA (RMSE: {rmse:.3f})',
281          color='red', linestyle='--')
282 plt.plot(test.index, forecast_test['yhat'].values,
283          label=f'Prophet (RMSE: {rmse_prophet:.3f})',
284          color='blue', linestyle='-.')
285 plt.title('Model Comparison: ARIMA vs Prophet')
286 plt.xlabel('Date')
287 plt.ylabel('Value')
288 plt.legend()
289 plt.grid(True, alpha=0.3)
290 plt.show()

```

Listing 28: Time Series Analysis with ARIMA and Prophet

8 Model Deployment

8.1 Model Serialization

Why Serialize Models? To save trained models for later use without retraining.

8.2 Model Monitoring

Why Monitor Models? Models can degrade over time due to:

- **Concept drift:** Relationships between features and target change
- **Data drift:** Feature distributions change
- **Performance decay:** Model becomes less accurate

```

1 import joblib
2 import pickle
3 import json
4 import numpy as np
5 import pandas as pd
6 from sklearn.ensemble import RandomForestClassifier
7 from sklearn.datasets import make_classification
8 from sklearn.model_selection import train_test_split
9 import torch
10 import torch.nn as nn
11

```

```

12 # 1. Create and train a model
13 X, y = make_classification(n_samples=1000, n_features=20, random_state=42)
14 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪ random_state=42)
15
16 model = RandomForestClassifier(n_estimators=100, random_state=42)
17 model.fit(X_train, y_train)
18
19 # 2. Save model using joblib (recommended for scikit-learn)
20 print("1. Saving model with joblib")
21 joblib.dump(model, 'random_forest_model.joblib')
22 print("Model saved as 'random_forest_model.joblib'")
23
24 # 3. Save model using pickle
25 print("\n2. Saving model with pickle")
26 with open('random_forest_model.pkl', 'wb') as f:
27     pickle.dump(model, f)
28 print("Model saved as 'random_forest_model.pkl'")
29
30 # 4. Save PyTorch model
31 print("\n3. Saving PyTorch model")
32 # Create a simple neural network
33 class SimpleNN(nn.Module):
34     def __init__(self, input_size, hidden_size, output_size):
35         super(SimpleNN, self).__init__()
36         self.fc1 = nn.Linear(input_size, hidden_size)
37         self.relu = nn.ReLU()
38         self.fc2 = nn.Linear(hidden_size, output_size)
39
40     def forward(self, x):
41         x = self.fc1(x)
42         x = self.relu(x)
43         x = self.fc2(x)
44         return x
45
46 pytorch_model = SimpleNN(20, 10, 2)
47 torch.save(pytorch_model.state_dict(), 'pytorch_model.pth')
48 print("PyTorch model saved as 'pytorch_model.pth'")
49
50 # Save entire model (architecture + weights)
51 torch.save(pytorch_model, 'pytorch_model_full.pth')
52 print("Full PyTorch model saved as 'pytorch_model_full.pth'")
53
54 # 5. Load models
55 print("\n4. Loading models")
56 loaded_model_joblib = joblib.load('random_forest_model.joblib')
57 print("Joblib model loaded successfully")

```

```

58
59 with open('random_forest_model.pkl', 'rb') as f:
60     loaded_model_pickle = pickle.load(f)
61     print("Pickle model loaded successfully")
62
63     # Load PyTorch model
64     loaded_pytorch_model = SimpleNN(20, 10, 2)
65     loaded_pytorch_model.load_state_dict(torch.load('pytorch_model.pth'))
66     loaded_pytorch_model.eval()
67     print("PyTorch model loaded successfully")
68
69     # 6. Test loaded models
70     print("\n5. Testing loaded models")
71     test_accuracy_joblib = loaded_model_joblib.score(X_test, y_test)
72     test_accuracy_pickle = loaded_model_pickle.score(X_test, y_test)
73
74     print(f"Joblib model accuracy: {test_accuracy_joblib:.4f}")
75     print(f"Pickle model accuracy: {test_accuracy_pickle:.4f}")
76
77     # 7. Save model metadata
78     print("\n6. Saving model metadata")
79     metadata = {
80         'model_type': 'RandomForestClassifier',
81         'n_features': X.shape[1],
82         'n_classes': len(np.unique(y)),
83         'training_date': '2024-01-01',
84         'accuracy': float(model.score(X_test, y_test)),
85         'parameters': model.get_params(),
86         'feature_names': [f'feature_{i}' for i in range(X.shape[1])]
87     }
88
89     with open('model_metadata.json', 'w') as f:
90         json.dump(metadata, f, indent=2)
91     print("Model metadata saved as 'model_metadata.json'")
92
93     # 8. Create prediction pipeline
94     print("\n7. Creating prediction pipeline")
95     class ModelPipeline:
96         def __init__(self, model_path, metadata_path=None):
97             """
98             Initialize model pipeline.
99
100             Args:
101                 model_path: Path to saved model
102                 metadata_path: Path to model metadata (optional)
103             """
104             self.model = joblib.load(model_path)

```

```

105
106     if metadata_path:
107         with open(metadata_path, 'r') as f:
108             self.metadata = json.load(f)
109     else:
110         self.metadata = None
111
112 def preprocess(self, X):
113     """
114     Preprocess input data.
115     Add any necessary preprocessing steps here.
116     """
117     # Example: Convert to numpy array if needed
118     if isinstance(X, pd.DataFrame):
119         X = X.values
120
121     # Add more preprocessing as needed
122     return X
123
124 def predict(self, X):
125     """
126     Make predictions.
127
128     Args:
129         X: Input features
130
131     Returns:
132         Predictions
133     """
134     X_processed = self.preprocess(X)
135     return self.model.predict(X_processed)
136
137 def predict_proba(self, X):
138     """
139     Make probability predictions.
140
141     Args:
142         X: Input features
143
144     Returns:
145         Probability predictions
146     """
147     X_processed = self.preprocess(X)
148     if hasattr(self.model, 'predict_proba'):
149         return self.model.predict_proba(X_processed)
150     else:
151         raise AttributeError("Model doesn't support predict_proba")

```

```

152
153     def get_metadata(self):
154         """
155         Get model metadata.
156         """
157         return self.metadata
158
159 # Test the pipeline
160 pipeline = ModelPipeline('random_forest_model.joblib', 'model_metadata.json')
161
162 # Make predictions
163 sample_data = X_test[:5]
164 predictions = pipeline.predict(sample_data)
165 probabilities = pipeline.predict_proba(sample_data)
166
167 print(f"\nSample predictions: {predictions}")
168 print(f"Sample probabilities:\n{probabilities}")
169 print(f"\nModel metadata:\n{json.dumps(pipeline.get_metadata(), indent=2)}")
170
171 # 9. Create API for model serving (Flask example)
172 print("\n8. Creating Flask API for model serving")
173 """
174 Create a file named 'app.py' with the following content:
175
176 from flask import Flask, request, jsonify
177 import joblib
178 import numpy as np
179 import pandas as pd
180
181 app = Flask(__name__)
182
183 # Load model
184 model = joblib.load('random_forest_model.joblib')
185
186 @app.route('/predict', methods=['POST'])
187 def predict():
188     try:
189         # Get data from request
190         data = request.get_json()
191
192         # Convert to numpy array
193         features = np.array(data['features']).reshape(1, -1)
194
195         # Make prediction
196         prediction = model.predict(features)
197         probability = model.predict_proba(features)
198

```

```

199         # Prepare response
200         response = {
201             'prediction': int(prediction[0]),
202             'probability': probability[0].tolist(),
203             'status': 'success'
204         }
205
206         return jsonify(response)
207
208     except Exception as e:
209         return jsonify({'error': str(e), 'status': 'error'})
210
211 @app.route('/health', methods=['GET'])
212 def health():
213     return jsonify({'status': 'healthy'})
214
215 if __name__ == '__main__':
216     app.run(debug=True, host='0.0.0.0', port=5000)
217
218 Run with: python app.py
219 """
220
221 # 10. Create Dockerfile for containerization
222 print("\n9. Creating Dockerfile for containerization")
223 """
224 Create a file named 'Dockerfile' with the following content:
225
226 # Use official Python runtime as base image
227 FROM python:3.9-slim
228
229 # Set working directory
230 WORKDIR /app
231
232 # Copy requirements file
233 COPY requirements.txt .
234
235 # Install dependencies
236 RUN pip install --no-cache-dir -r requirements.txt
237
238 # Copy application files
239 COPY . .
240
241 # Expose port
242 EXPOSE 5000
243
244 # Run application
245 CMD ["python", "app.py"]

```

```

246
247 Build and run:
248 docker build -t ml-api .
249 docker run -p 5000:5000 ml-api
250 """
251
252 # 11. Model monitoring
253 print("\n10. Model monitoring setup")
254 class ModelMonitor:
255     def __init__(self, model, window_size=100):
256         """
257         Monitor model performance over time.
258
259         Args:
260             model: Trained model
261             window_size: Size of sliding window for metrics
262         """
263         self.model = model
264         self.window_size = window_size
265         self.predictions_history = []
266         self.actuals_history = []
267         self.metrics_history = []
268
269     def update(self, X_batch, y_batch):
270         """
271         Update monitor with new batch of predictions.
272
273         Args:
274             X_batch: Features batch
275             y_batch: Actual labels batch
276         """
277         # Make predictions
278         y_pred = self.model.predict(X_batch)
279         y_pred_proba = self.model.predict_proba(X_batch)
280
281         # Store history
282         self.predictions_history.extend(y_pred)
283         self.actuals_history.extend(y_batch)
284
285         # Keep only window_size most recent
286         self.predictions_history = self.predictions_history[-self.window_size:]
287         self.actuals_history = self.actuals_history[-self.window_size:]
288
289         # Calculate metrics
290         if len(self.actuals_history) >= 10: # Minimum samples
291             from sklearn.metrics import accuracy_score
292             accuracy = accuracy_score(self.actuals_history, self.

```

```

293         ↪ predictions_history)
294
295     metrics = {
296         'accuracy': accuracy,
297         'n_samples': len(self.actuals_history),
298         'timestamp': pd.Timestamp.now()
299     }
300
301     self.metrics_history.append(metrics)
302
303 def get_current_metrics(self):
304     """
305     Get current metrics.
306     """
307     if self.metrics_history:
308         return self.metrics_history[-1]
309     return None
310
311 def plot_metrics_history(self):
312     """
313     Plot metrics over time.
314     """
315     import matplotlib.pyplot as plt
316
317     if not self.metrics_history:
318         print("No metrics history available")
319         return
320
321     df = pd.DataFrame(self.metrics_history)
322     df.set_index('timestamp', inplace=True)
323
324     plt.figure(figsize=(10, 6))
325     plt.plot(df.index, df['accuracy'], 'bo-')
326     plt.title('Model Accuracy Over Time')
327     plt.xlabel('Time')
328     plt.ylabel('Accuracy')
329     plt.grid(True, alpha=0.3)
330     plt.tight_layout()
331     plt.show()
332
333 # Test monitoring
334 monitor = ModelMonitor(model, window_size=50)
335
336 # Simulate batch updates
337 for i in range(0, len(X_test), 10):
338     X_batch = X_test[i:i+10]
339     y_batch = y_test[i:i+10]

```

```

339     monitor.update(X_batch, y_batch)
340
341 current_metrics = monitor.get_current_metrics()
342 print(f"\nCurrent model metrics: {current_metrics}")
343 print("\nModel monitoring is ready. Use monitor.plot_metrics_history() to
    ↪ visualize.")

```

Listing 29: Model Serialization and Deployment

9 Best Practices and Tips

9.1 Key Takeaways

Machine Learning Best Practices

1. **Always start with a baseline:** Simple model establishes performance floor
2. **Feature engineering & model selection:** Good features make learning easy
3. **Cross-validate everything:** Avoid overfitting to validation set
4. **Monitor for drift:** Models degrade as data changes
5. **Interpretability matters:** Understand model decisions
6. **Document everything:** Reproducibility is key
7. **Think about deployment:** Model needs to work in production
8. **Ethics first:** Consider bias, fairness, privacy

9.2 Choosing the Right Algorithm

Problem Type	Recommended Algorithms	When to Use	Considerations
Small Dataset	Logistic Regression, Linear SVM	Limited data, need interpretability	Risk of overfitting
Large Dataset	Random Forest, XG-Boost, Neural Nets	Plenty of data, complex patterns	Computational resources
Text Data	Naive Bayes, RNNs, Transformers	Natural language processing	Pre-trained embeddings help
Image Data	CNNs, Vision Transformers	Computer vision tasks	Transfer learning essential
Time Series	ARIMA, LSTM, Prophet	Sequential data, forecasting	Handle seasonality, trends
Anomaly Detection	Isolation Forest, One-class SVM	Fraud detection, system monitoring	Handle imbalanced data
Recommendation	Matrix Factorization, Neural CF	Personalized recommendations	Cold start problem

Table 8: Algorithm selection guide

9.3 Common Pitfalls and Solutions

Pitfall	Symptoms	Solutions
Overfitting	Train accuracy $\gg$ Test accuracy	Regularization, Cross-validation, Early stopping
Underfitting	Poor performance on both train/test	More complex model, Feature engineering
Data Leakage	Suspiciously high test accuracy	Proper train/test split, Time series splitting
Class Imbalance	Poor minority class recall	Resampling, Class weights, Different metrics
High Dimensionality	Slow training, Overfitting	Feature selection, Dimensionality reduction
Non-stationarity	Degrading performance over time	Regular retraining, On-line learning

Table 9: Common machine learning pitfalls and solutions

9.4 Performance Optimization Tips

1. **Feature Engineering:** Often more important than model choice
2. **Proper Validation:** Use cross-validation, avoid data leakage
3. **Ensemble Methods:** Combine multiple models for better performance
4. **Hyperparameter Tuning:** Systematic search for optimal parameters
5. **Regularization:** Prevent overfitting, improve generalization
6. **Early Stopping:** Stop training when validation performance plateaus
7. **Learning Rate Scheduling:** Adjust learning rate during training
8. **Gradient Clipping:** Prevent exploding gradients in deep learning
9. **Batch Normalization:** Stabilize training in neural networks
10. **Dropout:** Regularization technique for neural networks

9.5 Model Interpretability

```
1 import shap
2 import lime
3 import eli5
4 from sklearn.inspection import permutation_importance, partial_dependence
5 import matplotlib.pyplot as plt
6
7 # SHAP values for model interpretation
8 def explain_with_shap(model, X_train, X_explain):
9     """
10     Explain model predictions using SHAP.
11     """
12     # Create explainer
13     explainer = shap.TreeExplainer(model)
14
15     # Calculate SHAP values
16     shap_values = explainer.shap_values(X_explain)
17
18     # Summary plot
19     shap.summary_plot(shap_values, X_explain, show=False)
20     plt.title('SHAP Feature Importance')
21     plt.tight_layout()
22     plt.show()
23
24     # Force plot for individual prediction
25     shap.force_plot(
```

```

26         explainer.expected_value,
27         shap_values[0, :],
28         X_explain.iloc[0, :],
29         show=False,
30         matplotlib=True
31     )
32     plt.title('SHAP Force Plot for First Instance')
33     plt.tight_layout()
34     plt.show()
35
36     return shap_values
37
38 # Permutation importance
39 def permutation_importance_analysis(model, X, y):
40     """
41     Calculate permutation importance.
42     """
43     result = permutation_importance(
44         model, X, y,
45         n_repeats=10,
46         random_state=42,
47         n_jobs=-1
48     )
49
50     # Plot
51     sorted_idx = result.importances_mean.argsort()
52
53     plt.figure(figsize=(10, 6))
54     plt.boxplot(
55         result.importances[sorted_idx].T,
56         vert=False,
57         labels=[f'Feature {i}' for i in sorted_idx]
58     )
59     plt.title('Permutation Importance')
60     plt.xlabel('Importance')
61     plt.tight_layout()
62     plt.show()
63
64     return result
65
66 # Partial dependence plots
67 def partial_dependence_analysis(model, X, features):
68     """
69     Create partial dependence plots.
70     """
71     fig, axes = plt.subplots(1, len(features), figsize=(15, 5))
72

```

```

73     for idx, feature in enumerate(features):
74         pd_result = partial_dependence(
75             model, X, [feature],
76             kind='average'
77         )
78
79         axes[idx].plot(pd_result['values'][0], pd_result['average'][0])
80         axes[idx].set_title(f'Partial Dependence: {feature}')
81         axes[idx].set_xlabel(feature)
82         axes[idx].set_ylabel('Partial Dependence')
83         axes[idx].grid(True, alpha=0.3)
84
85     plt.tight_layout()
86     plt.show()
87
88 # LIME for local explanations
89 def explain_with_lime(model, X_train, instance, feature_names):
90     """
91     Explain individual predictions using LIME.
92     """
93     from lime.lime_tabular import LimeTabularExplainer
94
95     explainer = LimeTabularExplainer(
96         X_train,
97         feature_names=feature_names,
98         class_names=['Class 0', 'Class 1'],
99         mode='classification'
100     )
101
102     exp = explainer.explain_instance(
103         instance,
104         model.predict_proba,
105         num_features=10
106     )
107
108     # Show explanation
109     exp.show_in_notebook()
110
111     return exp
112
113 # Example usage
114 # Assuming 'model' is trained and 'X_train', 'y_train' available
115 # shap_values = explain_with_shap(model, X_train, X_train[:100])
116 # perm_result = permutation_importance_analysis(model, X_train, y_train)
117 # partial_dependence_analysis(model, X_train, ['feature_1', 'feature_2'])

```

Listing 30: Model Interpretability Techniques